Laboratoire O93 – conception expérimentale, variantes et critères de décision

AAM – dossier de recherche et d'implémentation

12 octobre 2026

Résumé

Ce document précède l'implémentation O93. Il transforme les objectifs de l'utilisateur, le formalisme O92-HFPN et les constats vérifiés de O92_0 en hypothèses testables, variantes d'architecture, protocoles et critères de sortie. L'objectif central est de remplacer l'assemblage de fonctions partiellement indépendantes par une boucle commune : demande/objet à traiter, observation, génération et composition de sondes, empreinte typée, sélection ou acquisition de compétence, action, vérification indépendante, retour de récompense/pénalité et mémoire persistante. Les résultats futurs devront être produits par les scripts, non préinscrits dans le modèle ni supposés à l'avance.

Table des matières

1 Description et cadre de laboratoire **3**
- 1.1 Question de recherche . . . 3
- 1.2 Description du modèle hérité O92_0 . . . 3
- 1.3 Règles rouges expérimentales . . . 3

2 Architecture conceptuelle candidate **5**
- 2.1 Boucle unifiée cible . . . 5
- 2.2 Variantes comparées . . . 5
- 2.3 Variantes du moteur de sondes . . . 6

3 Plan d'expériences de laboratoire **7**
- 3.1 E0 – cohérence du corpus, code et checkpoint . . . 7
- 3.2 E1 – observation universelle et provenance . . . 7
- 3.3 E2 – génération et sélection adaptative de sondes . . . 7
- 3.4 E3 – apprentissage source-off . . . 7
- 3.5 E4 – compétence composable . . . 8
- 3.6 E5 – mémoire et autonomie de fond . . . 8
- 3.7 E6 – RAG observation-first et routage . . . 8
- 3.8 E7 – modalité, combinaison et transformation . . . 8
- 3.9 E8 – interface, ressources et sûreté . . . 8

4 Matrice des mesures et règles de décision **9**
- 4.1 Préservation des données et analyse des erreurs . . . 9

5 Résultats connus, décisions et statut du laboratoire **10**
- 5.1 Journal d'exécution des gates O93 effectuées . . . 10
  - 5.1.1 P0 – fondations et intégrité . . . 10
  - 5.1.2 P1 – observation des fichiers et adaptateurs . . . 10
  - 5.1.3 P2 – moteur adaptatif de sondes . . . 11
  - 5.1.4 P3 – compétences de base et composition . . . 11
  - 5.1.5 P4 – acquisition adaptative et reproduction source-off . . . 12
- 5.2 Décision d'architecture avant implémentation . . . 13
- 5.3 Conséquences . . . 13
- 5.4 Commandes prévues et conventions . . . 13
- 5.5 Résultats de P5 et corrections en cours . . . 13

A Gabarit de rapport d'expérience **15**

B P6 – autonomie de fond, mémoire continue et boucle d'action locale **16**
- B.1 P6 – rejeu après correction de l'ordonnanceur UI et ajout de l'actionneur autonome . 17
  - B.1.1 P6 – preuve d'acceptation de l'interface réelle . . . 17
- B.2 Clôture de P6 et transition contrôlée vers P7 . . . 18

C P7 – intégration chat, génération, média et Observer **19**
C.1 P7 – rejeu intégré après normalisation de l'assertion . . . 19
C.2 P7 – essai réel de workflow de l'interface . . . 20
C.2.1 P7 feedback utilisateur – audit avant correction . . . 20
C.2.2 P7 feedback UI – implémentation et vérification de bout en bout . . . 21
C.3 Clôture de P7 et régression dépendante P0–P7 . . . 21
C.3.1 Transition P7–P8 – incident de copie de checklist, sans exécution du modèle . 22

D P8 – sécurité, intégrité et résilience (phase ouverte) **23**
D.0.1 P8 – première suite de sécurité : défaut de symlink détecté . . . 23
D.0.2 P8 – script de correction partiellement appliqué, sans test post-correctif . . . 24
D.0.3 P8 – suite adversariale initiale après corrections . . . 24
D.0.4 P8 – rejeu des phases dépendantes . . . 25
D.0.5 P8 – synthèse de sortie avant clôture . . . 25
D.1 Historique des protocoles P9 antérieurs et tentative stricte non concluante . . . 26
D.1.1 Tentative 01 – cinq graines et worker source-off : échec de mesure . . . 26
D.1.2 Historique de préinscription et protocole mesuré ensuite . . . 26

E P9 – essai exploratoire V1 à trois graines (historique ; remplacé par V2) **27**
E.1 Description . . . 27
E.2 Architecture . . . 27
E.3 Pipeline complet . . . 27
E.4 Commandes d'exécution . . . 27
E.5 Données, graines et intégrité . . . 28
E.6 Résultats bruts . . . 28
E.7 Interprétation des résultats . . . 29
E.8 Conséquences théoriques . . . 29
E.9 Conséquences conceptuelles . . . 29
E.10 Conséquences opérationnelles . . . 29
E.11 Erreurs, limites et preuves . . . 29
E.12 Conclusion et statut . . . 29

F Rapport consolidé de clôture des phases P8–P9 **31**

G P9 — audit post-entraînement et correction du vérificateur (état intermédiaire) **33**

H P9 — résultat final du protocole préenregistré V2 **35**
H.1 Rejeu du vérificateur post-run et tests unitaires V2 . . . 36
H.2 Régression combinée finale P0–P9 . . . 37
H.3 P10 – acceptation portable et gates environnementales ouvertes . . . 38

Chapitre 1

Description et cadre de laboratoire

1.1 Question de recherche

Comment un modèle AAM local-first peut-il acquérir et réutiliser une capacité depuis l'observation d'un objet, d'un programme, d'un dispositif, d'un mécanisme en boîte noire ou d'exemples entrée/sortie, tout en conservant l'observation, les compétences, la mémoire, le raisonnement, la recherche de données et la conversation dans un seul cycle de résolution ?

L'hypothèse de construction O93 est que les composants partagent un même contrat de tâche et une même provenance. Aucune hypothèse de réussite globale n'est posée. Chaque expérience caractérise un domaine, un coût, des données, une graine, un état de source/oracle et une règle d'acceptation. « Reproduire parfaitement » est défini opérationnellement comme l'absence d'erreur sur une matrice finie, déclarée et indépendante, complétée par des tests de transfert et d'abstention ; ce résultat ne permet jamais, à lui seul, d'affirmer la perfection sur tous les contextes possibles.

1.2 Description du modèle hérité O92_0

O92_0 contient un moteur de signatures typées multimodales, un FPN dense entraînable, un petit LM compact entraînable, des capsules de compétences exportables, une boucle d'objectif, une mémoire événementielle, un index RAG lexical et une interface Tkinter. L'audit O92_0 a montré des points de jonction incomplets : le LM n'est pas le générateur par défaut du chat ; le RAG est lexical et n'apprend pas automatiquement une compétence générale à partir d'un texte non étiqueté ; les données synthétiques multimodales sont trop réduites pour une conclusion de généralisation ; l'autonomie n'a pas encore une boucle de fond connectée à toutes les fonctions ; l'observation matériel/capteurs est incomplète.

O93 conserve les implémentations utilisables comme adaptateurs historiques, mais fait passer les fonctionnalités par le planificateur commun et journalise le chemin réellement suivi. Une ancienne recherche ou un ancien classificateur peut être sélectionné comme compétence parmi d'autres, pas présenté comme le modèle entier.

1.3 Règles rouges expérimentales

1. Ne jamais enregistrer un verdict prédéfini sur le potentiel général d'AAM.
2. Ne jamais confondre mécanisme codé, mécanisme exécuté, résultat mesuré et capacité indépendante validée.
3. Ne jamais inscrire une sortie attendue dans le solveur comme si elle venait du modèle ; les réponses-cibles ne servent qu'au contrôle indépendant et sont absentes du chemin d'inférence.
4. Les entrées sont non fiables. Un programme observé par AST est inspecté statiquement ; il n'est pas exécuté pendant l'observation.

5. Les actions de caméra/capteur sont explicites, visibles et révocables. Aucun flux physique n'est capturé en secret.
6. La mémoire de raisonnement peut être coupée sans arrêter la boucle autonome ; le journal d'audit minimal de sécurité et d'intégrité reste distinct de cette mémoire facultative.
7. Les budgets bornent une exécution, jamais la capacité déclarée du modèle. Toute interruption donne un checkpoint et un statut de ressource.
8. Un essai en échec est conservé et relié au correctif ; le holdout ne choisit pas les hyperparamètres.

Chapitre 2

Architecture conceptuelle candidate

2.1 Boucle unifiée cible

1. **Entrée universelle** : texte, JSON, fichier, dossier, image, audio, vidéo, flux de capteur, caméra autorisée, résultat d'un outil ou objectif autonome.
2. **Contrat de tâche** : objectif, type d'entrée/sortie attendu, contexte, droits, préconditions, critères de réussite, budget, sources et politique de mémoire.
3. **Observation** : inspection du transport, du contenu décodable, des métadonnées, des états d'erreur et de la provenance. Les octets originaux sont conservés ou référencés par hash.
4. **Sondage** : réutiliser les sondes ; composer des sondes typées ; tester des miroirs/références ; analyser les collisions résiduelles ; ne créer une primitive qu'après hypothèse mesurable.
5. **Empreinte** : assembler les valeurs et leurs états, la hiérarchie globale/locale/temporelle, les modalités, les unités, la confiance et l'identité de la sonde.
6. **Routage des compétences** : sélectionner des compétences prédéfinies, entraînées, importées, apprises depuis les traces ou composées. Chaque compétence est associée à un domaine et à ses preuves.
7. **Action/génération** : transformer, classifier, générer du texte, répondre à partir de sources, composer une procédure ou proposer une expérience. Les actions à effet externe traversent les contrôles de permission.
8. **Vérification indépendante** : exécuter les sondes de sortie, vérifier le contrat, comparer à une référence seulement si disponible et autorisée, évaluer généralisation, coût et abstention.
9. **Récompense/pénalité** : utiliser la progression, la vérification, l'utilité notée par l'utilisateur, la nouveauté et le coût. La récompense ne confère aucune permission et ne remplace pas la preuve.
10. **Mémoire et reprise** : enregistrer l'épisode, les poids, le résultat, l'erreur, la sonde essayée et le checkpoint selon la politique de conservation.

2.2 Variantes comparées

Variante	Mécanisme	Avantage à mesurer	Risque / critique
A – O92 modulaire	Conserver les services actuels et ajouter des raccords directs par fonctionnalité.	Coût minimal, réutilisation rapide.	Les fonctionnalités restent potentiellement séparées ; risque d'incohérence des états et politiques.

B – pipeline unifié strict	Tout passe par un contrat de tâche commun, un observateur, un orchestrateur, un vérificateur et un journal.	Traçabilité, tests de bout en bout, sélection des compétences cohérente.	Coût d'orchestration et contrat trop rigide s'il ne permet pas d'étendre les types.
C – graphe événementiel ouvert	Chaque compétence publie des événements et peut proposer des sondes/actions dans un graphe partagé.	Extensibilité, composition de compétences et apprentissage de nouvelles familles.	Plus complexe à auditer ; nécessite cycles, conflits et permissions contrôlés.

Décision de conception provisoire : une combinaison B+C : contrat commun pour chaque requête, graphe de compétences extensible et journal d'événements vérifiable. La décision est fondée sur les exigences d'intégration et d'audit, pas sur une prétendue mesure de performance non exécutée.

2.3 Variantes du moteur de sondes

S0 – sondes fixes Un ensemble immuable de sondes multimodales. Baseline simple, ne peut pas répondre à une collision nouvelle sans intervention humaine.

S1 – composition adaptative Recherche d'abord les sondes existantes, les compositions valides et les sondes miroir ; compare ensuite les paires confondues. Génère des candidates paramétrées.

S2 – propositions apprises En plus de S1, apprend à estimer le gain de séparation/coût d'une sonde et choisit la prochaine sonde. Un vérificateur indépendant mesure le gain.

Le parcours minimal retenu pour l'implémentation commence par S1. S2 n'est promu qu'avec un dataset de collisions annotées ou une métrique proxy indépendante ; aucune valeur n'est présumée.

Chapitre 3

Plan d'expériences de laboratoire

Chaque expérience comporte description, architecture, pipeline, commandes, résultats, interprétation et conséquences théorique/conceptuelle/opérationnelle. Avant l'exécution, son plan et son hash sont enregistrés. Chaque sortie renvoie à la graine, au corpus et à la version du code.

3.1 E0 – cohérence du corpus, code et checkpoint

Question : le point de départ O92_0 est-il intégral, reproductible et identifiable ? **Pipeline** : vérifier les ZIP et manifestes ; extraire ; calculer SHA-256 ; analyser les checklists/erreurs/formalisme ; compiler Python ; enregistrer les faits connus et les lacunes. **Commandes** (depuis le dossier d'audit) : `python -m compileall -q aam_o92 tests` ; `python scripts/run_gates.py -help` si disponible ; `python -m unittest discover -s tests -v`. **Critère** : aucune archive lue sans vérification ; tous les fichiers audités se trouvent dans le manifeste ; les états hérités sont conservés et les checklists O93 commencent sur un nouveau checkpoint. Une anomalie bloque la phase 0 jusqu'à correction ou qualification comme indisponibilité externe.

3.2 E1 – observation universelle et provenance

Données : exemples locaux texte, JSON, JSONL/CSV, Python source, PNG/BMP, WAV, vidéo disponible et fichier binaire inconnu. **Contrôles** : fichier connu, entrée mal formée, vide, encodage non UTF-8, fichier surdimensionné, lien symbolique, chemin Unicode, dossier contenant des sous-dossiers, dispositif absent. **Métriques** : fichiers traités, exactitude des types détectés, hash, états de decode, temps, mémoire, taux d'erreurs attendues et absence de modification des sources. **Acceptation** : chaque entrée obtient un résultat explicite 'VALID', 'PARTIAL', `NOT_DECODED`, 'UNSUPPORTED', 'ERROR' ou 'TIMEOUT' ; pas d'interprétation sémantique fictive pour un binaire non décodé.

3.3 E2 – génération et sélection adaptative de sondes

Construire des paires contrôlées et collisionnelles, garder des paires de test séparées, exécuter S0/S1 et mesurer collision initiale, collisions restantes, paires séparées, faux séparations de véritables invariants, stabilité, coût et nombre de sondes. Les sondes miroir et composées précèdent toujours la création de primitives. L'algorithme doit s'arrêter par critère de couverture sur le domaine, épuisement d'un budget ou demande de revue ; l'arrêt budgétaire ne s'appelle pas échec de capacité.

3.4 E3 – apprentissage source-off

Créer des traces entrée/sortie à partir de mécanismes distincts : transformations exactes, classification synthétique, programme pur observé en AST plus exemples comportementaux, et

adaptateur simple. Partitionner par mécanisme/graine avant l'entraînement. Comparer mémoire de cas, FPN de base, FPN conditionné par fiabilité, compositions et candidat généré. Après acquisition, supprimer l'oracle de l'environnement d'inférence puis redémarrer le processus ; évaluer exactitude, couverture, calibration, comportement hors domaine, latence et poids persistants. Ne pas injecter les sorties-cibles dans le chemin runtime.

3.5 E4 – compétence composable

Acquérir les briques 'observer', 'fingerprint', 'probe-generation', 'orchestrator' et 'logical-reasoning' ; tester chaque brique seule, toutes les paires compatibles et les compositions de trois briques. Un contrat de composition déclare type, effet, pré/postcondition, risque, coût et vérificateur. Mesurer si une composition est réutilisable sur une tâche différente de celle de l'acquisition.

3.6 E5 – mémoire et autonomie de fond

Lancer le runtime sans donnée utilisateur. Vérifier que la boucle s'exécute, inspecte son propre état et son inventaire, élabore une prochaine observation limitée, et conserve un journal continu tant que la mémorisation est active. Désactiver uniquement l'enregistrement du contenu de raisonnement et vérifier que l'orchestration reste vivante, tandis que la mémoire déclarative/raisonnement ne croît plus. Stopper l'autonomie doit arrêter le runtime utilisateur et préserver checkpoint/reprise. Tester double lancement, verrouillage, fermeture forcée et arrêt propre.

3.7 E6 – RAG observation-first et routage

Comparer quatre modes : désactivé ; recherche lexicale explicite ; observation seule ; observation + acquisition supervisée quand un manifeste contient des paires d'exemples. Utiliser des documents sans labels, documents avec Q/R, documents contradictoires et injection de prompt. Mesurer les citations exactes, support des affirmations, taux d'abstention, gain/fuite de données et temps. Les textes sans labels produisent une mémoire de contenu et des hypothèses, pas de pseudo-étiquettes inventées. La recherche classique est une compétence sélectionnable ; le mode normal observe les sources puis choisit les compétences étayées.

3.8 E7 – modalité, combinaison et transformation

Image : vues globale, régions, image miroir/inversée, silhouette, échelles et orientations. Audio : signal global, segments, énergie, spectre, transformations temporelles contrôlées. Vidéo : frames, timestamps, mouvement et synchronisation ; si les dépendances manquent, `NOT_RUN` reste explicite. Tester l'invariance seulement quand elle est attendue par le contrat ; pour rotation, inversion, vitesse audio ou changement de scène, mesurer l'effet au lieu de présumer l'invariance.

3.9 E8 – interface, ressources et sûreté

Tester UI Windows-native sur Windows 10/11, compte non-administrateur, chemins Unicode, CPU seul, ressources limitées, écriture impossible, données corrompues, autorisation de caméra, arrêt d'urgence, accès Internet désactivé puis allowlistée, et retour à une version antérieure. Un test Linux/Xvfb n'est pas une acceptation Windows. Les caméras et capteurs ne s'ouvrent jamais implicitement.

Chapitre 4

Matrice des mesures et règles de décision

Famille	Métriques minimales	Décision autorisée
Observation	Intégrité hash, couverture, statut par sonde, taux d'erreurs, coût.	Décoder/partiellement décoder/ne pas prétendre comprendre.
Sondes	Collision résiduelle, séparation des paires utiles, invariants cassés, coût.	Promouvoir uniquement si gain sur validation et contrôle holdout.
Compétence	Score de tâche, calibration, source-off, reprise, transfert et abstention.	CANDIDATE / PASS_BOUNDED / FAIL_QUALITY_GATE / UNRESOLVED.
Composition	Contrats, compatibilité types, préconditions, vérification, effets secondaires.	Composer si chaque arête du graphe satisfait le contrat.
RAG	Exactitude des citations, support, rappel/precision si vérité terrain, stabilité.	Utiliser en mode lexical ou autre mode explicitement évalué.
Autonomie	Pulses, tâches résolues, tâches abandonnées, erreurs, mémoire, arrêt/reprise.	Fonctionne dans le périmètre autorisé et peut être arrêté/repris.

4.1 Préservation des données et analyse des erreurs

Chaque run crée `run.json`, `events.jsonl`, `metrics.json`, `errors.jsonl`, la liste de fichiers/-sha256 et un checkpoint avant et après une phase. Les sources d'origine sont en lecture seule par défaut. Les générateurs d'images/audio d'essai doivent sauver manifeste, graines et paramètres. Les échecs ont des catégories : défaut du modèle, données, environnement, protocole, budget, intégration, sécurité ou test lui-même. Une correction entraîne une nouvelle version de run sans écraser les observations précédentes.

Chapitre 5

Résultats connus, décisions et statut du laboratoire

À l'ouverture du laboratoire, les seules mesures héritées acceptables sont celles déjà conservées dans O92_0 : petits essais synthétiques de classification, LM très compact, récupération lexicale locale et tests unitaires/gates de O92_0. Elles ne préjugent pas du comportement de O93. Les scores OOD image/audio très peu échantillonnés ne sont pas des preuves robustes de généralisation. Les résultats O93 seront ajoutés uniquement après exécution du protocole.

5.1 Journal d'exécution des gates O93 effectuées

5.1.1 P0 – fondations et intégrité

Description/architecture : contrats `TaskEnvelope/TaskResult`, prévol documentaire, journal d'événements append-only à chaîne de hash, mémoire de raisonnement facultative et checkpoints vérifiés. **Pipeline/commandes** : consultation du formalisme et du laboratoire, des checklists O93 et des registres d'erreurs hérités/actuels ; compilation de tous les modules ; exécution de `python -m unittest tests.test_p0_foundation -v`. **Résultats** : 6/6 tests passés sur la tentative finale. Un premier rejeu a échoué à cause d'une assertion de test qui attendait un champ absent du contrat public de `preflight` ; l'assertion a été corrigée, l'échec est conservé dans les journaux et l'erreur O93-E006 est résolue. Les registres d'erreurs O92 à la racine ont été conservés octet pour octet ; les erreurs cumulées O93 résident dans `o93_control/`. **Interprétation** : les contrats, le contrôle d'intégrité et le préflight sont opérationnels dans l'environnement Python de ce run ; ce résultat ne valide pas encore la capacité générale. **Conséquences** : théorique – séparation de l'événement de raisonnement de l'audit ; conceptuelle – un checkpoint corrompu est rejeté ; opérationnelle – reprise par phase documentée. **Statut** : `PASS_BOUNDED` ; checkpoint `P0_complete.json`.

5.1.2 P1 – observation des fichiers et adaptateurs

Description/architecture : observation hiérarchique via `O93Observer` au-dessus du moteur d'empreintes hérité, avec type source, hash, statut de décodage, AST, limites d'entrée et adaptateurs de capteurs soumis à autorisation. **Pipeline/commandes** : compilation puis `python -m unittest tests.test_p1_observer -v` ; l'essai combiné P0+P1 a également passé 17/17. **Résultats** : P1 passe 11/11 tests : texte, JSON, JSONL, CSV et TSV ; CSV à colonnes incohérentes rapporté en erreur ; parse AST Python sans exécution ; PNG et WAV locaux décodés par leurs voies de base ; fichier binaire inconnu marqué `NOT_DECODED` ; taille excessive refusée ; symlink ignoré/refusé ; adaptateur mock exigeant une autorisation explicite et toujours fermé ; API caméra qui n'ouvre rien pendant la découverte et refuse une capture sans autorisation. **Interprétation** : observation de ces formats et de l'adaptateur simulé est démontrée. Le matériel caméra réel n'a pas été ouvert et son essai est `NOT_RUN` ; aucune capture matérielle n'est revendiquée. **Conséquences** : théorique – l'empreinte binaire ne signifie pas décodage sémantique ; conceptuelle – les états inconnus restent visibles ;

opérationnelle – limites, erreurs, permissions et provenance sont testables. **Statut** : `PASS_BOUNDED` ; checkpoint `P1_complete.json`.

5.1.3 P2 – moteur adaptatif de sondes

Description/architecture : `ProbeFactory` ordonne les sondes existantes, les compositions, les sondes miroir avec référence, le diagnostic de collisions, puis des probes de taille/forme élargies par rounds. L'état contient le catalogue, les probes déjà tentées, les rounds et un hash de checkpoint. **Pipeline/commandes** : `python -m unittest tests.test_p2_probes -v` ; expérience `P2-shapes-train-001` sur uniquement les huit lignes `TRAIN` de `examples/datasets/o92_demo_shapes.jsonl` ; aucun accès à VALID/HOLDOUT/OOD pendant la sélection. **Résultats du test mécaniste** : 9/9 tests passés après l'ajout du score de similarité typée et de classement par paires ; ordre des stades, composition, miroir/référence, collisions, génération multi-échelle, checkpoint/reprise/anti-altération, budget nul et séparation entre exécution des sondes et évaluation post-hoc des labels. **Résultats de l'expérience** : le premier score basé sur des hashes exacts a été rejeté lors de l'audit parce qu'il confondait différence de contenu et fausse séparation. Avec similarité typée et classement par paires, 1 round TRAIN-only a proposé `text.word_frequency` : score=0.9932, séparation utile=1.000, rang pairwise=0.9948, dissimilarité moyenne intra-cible=0.6011, inter-cible=0.9772, marge=0.3761. Le taux de faux découpage brut reste 1.00 au seuil de dissimilarité 0.25 ; il est publié comme diagnostic et n'est pas transformé en verdict de capacité. Statut : `CANDIDATE_FOUND_NOT_GENERALIZATION_PROOF`. **Interprétation** : les features lexicales permettent de classer les paires inter-cibles au-dessus des paires intra-cible dans les huit exemples TRAIN. Le taux de faux découpage reste élevé parce que les variantes d'un même label ne partagent pas un vecteur exact ; un réseau entraînable peut apprendre sur des représentations variables, mais seule la validation de la compétence en aval établit si cela fonctionne. La candidate n'est pas promue automatiquement ; P4 vérifie entraînement réel, capsule, replay source-off et partitions indépendantes. **Conséquences** : théorique – les hashes restent des preuves d'identité/provenance, tandis que la similarité typée mesure une ressemblance graduée ; conceptuelle – un candidat de sonde doit apporter un signal inter-cible supérieur à sa variabilité intra-cible ; opérationnelle – score pairwise, marge, faux découpage, budget et checkpoint sont enregistrés distinctement. **Statut** : moteur P2 `PASS_BOUNDED` sur les tests et l'expérience TRAIN-only ; candidat de sonde `text.word_frequency` découvert mais non certifié comme compétence ; la généralisation reste un gate séparé. Checkpoint : `checkpoints/P2_adaptive_search_checkpoint.json`.

5.1.4 P3 – compétences de base et composition

Description/architecture : le registre possède les primitives `observe`, `create_fingerprint`, `create_probes`, `orchestrate` et `logical_reasoning`, complétées par `compose`, `verify` et des adaptateurs de génération/recherche explicitement optionnels. Chaque capacité annonce son contrat d'entrée/sortie, ses effets et un statut d'invocation séparé d'une certification générale. **Pipeline/commandes** : `python -m unittest tests.test_p3_competencies -v` ; régression en chaîne `tests.test_p0_foundation tests.test_p1_observer tests.test_p2_probes tests.test_p3_competenci` **Résultats** : 8/8 tests P3 et 33/33 tests cumulés P0–P3 passés. La fermeture logique a dérivé des faits par règles fournies avec trace de preuve ; la composition `create_fingerprint -> create_probes` a fonctionné et conservé les hashes par étape. Une composition de liste de probes vers génération textuelle a été arrêtée par non-conformité de type. Les checks de vérification PASS/FAIL sont séparés du signal de récompense utilisateur. **Interprétation** : les primitives sont exécutables et le composeur refuse le mauvais type, mais la logique formelle couverte ici est le forward chaining de règles Horn fournies par l'appelant, pas une preuve de raisonnement général. **Conséquences** : théorique – la composition utilise des artefacts structurés au lieu de transmettre aveuglément un wrapper ; conceptuelle – le résultat d'une étape est toujours vérifiable avant promotion ; opérationnelle – les erreurs d'interface restent des statuts explicites. **Statut** : `PASS_BOUNDED` ; pas de promotion automatique d'une compétence. Défaut de sérialisation initial O93-E008 corrigé et conservé au journal.

5.1.5 P4 – acquisition adaptative et reproduction source-off

Description/architecture : `O93AcquisitionService` relie le générateur adaptatif `ProbeFactory`, un décorateur `ProbeAugmentedFingerprintEngine`, le vectoriseur d'empreintes et le réseau FPN entraînable hérité d'O92. La capsule conserve les poids, le schéma de caractéristiques, les définitions/version des probes choisies et leur provenance, pas les exemples bruts. Le même adaptateur de sondes est chargé à l'inférence, de sorte que le vecteur vu après apprentissage respecte le contrat du vecteur entraîné. La promotion est séparée et n'est pas automatique. **Pipeline complet** : lecture du JSONL et vérification SHA-256 ; respect des partitions explicites ; génération et comparaison des sondes sur TRAIN uniquement ; classement post-hoc des résultats TRAIN sans injecter de cibles dans l'exécution des probes ; augmentation des caractéristiques ; entraînement Adam avec arrêt/sélection sur VALID ; écriture atomique de la capsule ; recalcul et vérification du hash ; rechargement dans un adaptateur nouvellement construit ; inférence où seul le champ input est fourni ; jonction des cibles par l'évaluateur après les prédictions ; mesure séparée de VALID, HOLDOUT, OOD et de l'abstention. **Commandes exécutées** : `PYTHONPATH=. python -m unittest tests.test_p4_acquisition -v` ; `PYTHONPATH=. python scripts/run_p4_experiment.py` ; la régression cumulée P0–P4 est exécutée après mise à jour de cette section et consignée dans le rapport final de gate. **Données/environnement** : `examples/datasets/o92_demo_shapes.jsonl`, SHA-256 `d6c75b2b06f566ee9afbbb7a8f902c04e87a2bad09d25`[illegible] partitions déclarées TRAIN=8, VALID=2, HOLDOUT=2, OOD=2 ; Python 3.13, exécution locale CPU. **Résultats bruts** : entraînement 40 époques, Adam, 7 742 paramètres de poids, loss de validation minimale $1.0472059266 \times 10^{-4}$, temps d'entraînement enregistré 2.698 s. Accuracy et balanced accuracy VALID=1.00 (n=2), HOLDOUT=1.00 (n=2), OOD=1.00 (n=2). La capsule de la première exécution a été conservée comme historique ; la capsule de l'exécution officielle corrigée est `trained_artifacts/o93_skill_candidates/skill-1791840630-33dfb09657.skill.json`, SHA-256 `69b3e64a53177ab31f62f9411115073f6402b05ed71314400c0a2748bceed428`. Le test de confiance avec seuil 0.999999 a répondu `ABSTAIN_LOW_CONFIDENCE`, sans inventer de décision acceptée. **Essai adaptatif de probes** : trois rounds, 26, 10 et 7 candidates évaluées. Meilleur candidat de développement déclaré : `length`, séparation utile=1.000, faux découpage intra-cible=0.9167, score=0.0827333. La condition n'est pas atteinte ; le statut `UNRESOLVED_BUDGET` et le motif `ROUND_BUDGET_EXHAUSTED` sont préservés. Les IDs `text.profile.character_class_profile.r2`, `text.char_ngrams.n96`, `collision.matrix.round2` et `text.char_ngrams.n128` sont intégrés à la capsule pour tester leur contribution, mais ils ne sont pas qualifiés comme probes optimales. **Interprétation** : ces résultats prouvent que l'acquisition produit un FPN entraîné qui est rechargé et rejoue une compétence sur les formats textuels du mini-corpus sans consulter le dataset au moment de l'inférence. Trois échantillons d'évaluation de taille deux ne justifient aucune conclusion statistique générale. Le score de longueur a un faux découpage très élevé ; la recherche adaptative n'a pas résolu toutes les collisions et reste réouvrable. Un nouveau texte libre `sharp polygon with straight edges` a été rendu comme `circle` ; comme aucune étiquette n'a été fournie à cette inférence, cette observation n'est pas comptée dans l'accuracy, mais révèle un cas candidat à tester indépendamment. **Erreurs et corrections** : la première tentative du test P4 n'a pas démarré à cause d'une chaîne Python mal échappée (O93-E009) ; la deuxième a reproduit l'`IndexError` hérité d'un helper qui ignorait la colonne cible personnalisée (O93-E010) ; la troisième a échoué sur une assertion de test qui traitait un vectoriseur matérialisé comme un dictionnaire (O93-E011) ; le rapport initial omettait les mesures par round par lecture du mauvais niveau de l'objet (O93-E012). Ces quatre entrées sont conservées et marquées résolues après correction et répétition. **Conséquences** : théorique – les sondes deviennent des coordonnées effectives de l'entrée neuronale au lieu d'un rapport indépendant ; conceptuelle – une compétence candidate est l'ensemble poids+vectoriseur+adaptateur de sondes+provenance ; opérationnelle – une capsule rechargée travaille sans la source et peut abstentionner, mais une promotion exige davantage de données, de seeds, de transfert et une évaluation indépendante. **Statut** : mécanisme P4 `PASS_BOUNDED` ; compétence acquise `CANDIDATE_PENDING_INDEPENDENT_ACCEPTANCE` ; pas de certification de généralisation. Rapport source : `logs/P4_official_experiment.json` ; checkpoint : `checkpoints/P4_complete.json`.

5.2 Décision d'architecture avant implémentation

Retenir la boucle unifiée B+C, un contrat partagé pour chaque tâche, un graphe de compétences versionné et une mémoire à hash-chain. Utiliser un portfolio de sondes extensible par paramètres/composeurs, commencer par sondes existantes et miroirs, mesurer le gain indépendant, et préserver les états `NOT_DECODED/NOT_RUN/UNRESOLVED`. La recherche lexicale reste un module optionnel. L'observation des sources RAG est le chemin par défaut ; l'apprentissage n'est lancé que si un signal d'entraînement exploitable existe ou si une expérience non supervisée a un critère indépendant mesurable.

5.3 Conséquences

Théorique : un même objet devient une collection de vues typées avec limites d'inférence visibles. La croissance du portefeuille de sondes ne donne un gain réel que si elle sépare des collisions pertinentes et survit à une contre-épreuve. **Conceptuelle** : une compétence est un artefact comportemental vérifié, et pas une ligne de mémoire ni une réponse retrouvée. Le graphe unifie la sélection et la composition sans affirmer que toute opération disponible est une intelligence générale. **Opérationnelle** : tout service devient inspectable, reprenable, mesurable et désactivable indépendamment. La boucle d'autonomie démarre avec l'application, tandis que les actions externes et dispositifs demandent une autorisation claire.

5.4 Commandes prévues et conventions

```
python -m compileall -q aam\_o92 aam\_o93 tests
python -m unittest discover -s tests -v
python -m aam\_o93 status
python -m aam\_o93 observe path/to/file
python -m aam\_o93 solve "decrire et traiter cet objet"
python -m aam\_o93 probes --domain text
python -m aam\_o93 autonomy start
python -m aam\_o93 ui
```

Les commandes qui ne sont pas livrées dans le checkpoint courant demeurent des commandes-cibles et sont marquées `NOT_IMPLEMENTED` dans le manifeste de phase ; leur mention ne prétend pas qu'elles ont déjà passé un essai.

5.5 Résultats de P5 et corrections en cours

Première gate P5 exécutée avec six tests. Deux assertions n'ont pas passé et la destruction de l'espace temporaire avant l'arrêt du runtime a produit six erreurs de teardown. L'analyse a isolé trois éléments distincts : (i) couverture incomplète du détecteur lexical d'injection pour une paraphrase, alors que les documents restent non fiables par défaut ; (ii) jeu de test supervisé mal construit, n'ayant qu'une seule classe dans TRAIN, rejeté à juste titre par le moteur ; (iii) ordre de nettoyage du banc de test. La phase reste RUNNING et ne progresse pas. Les corrections et le rejeu seront ajoutés comme suite du présent résultat.

Second essai P5 : 5/6 tests ont passé, sans erreur de fermeture. Les scénarios observation-first, citation, routage, plugin lexical manuel, détection de contenu non fiable et acquisition O93 ont fonctionné. Le seul échec est une assertion littérale qui attendait « targets are never invented » alors que la réponse conforme indique « labels not invented ». La checklist demeure P5/RUNNING ; l'assertion sera normalisée sémantiquement puis tous les tests P5 seront rejoués.

Troisième essai P5 : les six tests passent après correction de la couverture heuristique d'injection, du jeu d'apprentissage (deux classes présentes dans TRAIN), de l'ordre de nettoyage du banc et de l'assertion de contrat. Le rapport reproductible est `logs/P5_test_report.json` ; journal brut :

`logs/P5_test_knowledge_final.txt`. Le RAG normal reste une recherche lexicale BM25-like sur chunks observés ; aucune recherche sémantique neuronale n'est revendiquée. P5 demeure en attente de la régression complète P0-P5 avant fermeture.

Régression P0–P5 : les 46 tests ont tous affiché `ok` et le runner a écrit `Ran 46 tests ... OK`, en 25,045 s de temps de test ; néanmoins, la commande enveloppe n'a pas rendu un code de sortie confirmé avant son délai. L'acceptation de la régression est donc suspendue à une vérification de terminaison du banc (threads/processus/handles), sans effacer le résultat des assertions. Voir `logs/P0_P5_full_regression_timeout_summary.json`.

Diagnostic de terminaison après l'essai P0–P5 : le banc de test a été exécuté via un runner Python borné (sans pipeline `tee`). Les 46 tests affichent `OK`, le processus retourne le code 0 et le seul thread restant avant sortie est `MainThread`. Le précédent timeout de la commande enveloppe n'a pas été reproduit ; sa cause initiale précise reste indéterminée. Le rapport et la checklist enregistrent cette distinction. Les copies embarquées du laboratoire/formalisme ont aussi été détectées comme désynchronisées ; elles sont réalignées sur les sources canoniques avant la régression finale P0–P5.

Clôture P5 après rejeu post-synchronisation : la gate unitaire passe 6/6 et la régression complète P0–P5 passe 46/46. Le runner Python, exécuté sans pipeline intermédiaire, retourne explicitement le code 0 ; aucune erreur/exception n'est rapportée et seul `MainThread` est actif au retour du runner. P5 est donc `COMPLETE/PASS_BOUNDED` pour son périmètre déclaré. Cela prouve l'observation des contenus supportés, le stockage de chunks empreintés, la sélection optionnelle du plugin lexical et l'acquisition sur paires explicites dans les fixtures ; cela ne prouve ni recherche sémantique neuronale ni généralisation générale. La phase suivante est P6 : autonomie persistante, priorité issue du feedback, toggle mémoire et protection du lancement en double.

Annexe A

Gabarit de rapport d'expérience

Chaque expérience est rapportée avec les sections suivantes, dans cet ordre : Description ; Architecture ; Pipeline complet ; Commandes d'exécution exactes ; Environnement/données/graines/hashes ; Résultats bruts ; Interprétation ; Conséquences théoriques ; Conséquences conceptuelles ; Conséquences opérationnelles ; Erreurs et limites ; Conclusion ; Statut et prochaine action. Toute mesure doit pointer vers un artefact reproductible.

Annexe B

P6 – autonomie de fond, mémoire continue et boucle d'action locale

Description : la boucle autonome démarre depuis le runtime et l'interface. À chaque pulse, elle observe son état, produit un agenda, exécute un petit nombre d'actions locales autorisées, journalise l'issue et garde un checkpoint à l'arrêt. La mémoire d'audit est toujours active ; la mémoire narrative de raisonnement est commandable séparément.

Architecture : `AutonomousLoop` détient un verrou exclusif de workspace avec PID et token ; un second runtime n'est pas autorisé à prendre la main ni à arrêter le propriétaire. Un lock illisible est maintenant conservé et renvoie `LOCK_UNREADABLE_MANUAL_REVIEW`, car son illisibilité ne prouve pas la mort du propriétaire. `AAMRuntime.autonomy_step` réalise des actions sûres bornées : empreindre un instantané interne, contrôler l'intégrité des journaux, inspecter le manifeste de connaissances, différer l'apprentissage sans labels et créer des sondes de diagnostic pour une tâche insatisfaite lorsque son entrée est disponible. Aucune caméra, action externe ou connexion réseau n'est déclenchée par la boucle.

Pipeline/commandes : avant la suite, le préflight a relu le formalisme, le laboratoire, la checklist et les deux registres O92/O93. `python -m unittest tests.test_p6_autonomy -v` a passé 7/7 tests avant l'intégration de l'exécuteur d'actions. Ces tests démontrent l'auto-observation, le toggle mémoire, le retour utilisateur sans vérité fabriquée, le stop/checkpoint, le redémarrage, l'idempotence et le refus d'un double lancement. Une première ouverture GUI Xvfb a expiré ; la trace a isolé un double appel de statut dans le constructeur qui déclenchait une boîte modale quand une tâche de statut était déjà active. Ce défaut est enregistré O93-E020. La correction supprime le lancement de statut prématuré dans le builder et fait échouer l'arrêt UI de manière visible plutôt que de masquer un timeout.

Résultats actuels : P6 reste `RUNNING`, et non `COMPLETE`, jusqu'au rejeu de la régression P0–P6 terminant explicitement avec code 0 et aux contrôles de cohérence des logs/checkpoints. Le test minimal de Tk sous Xvfb a bien ouvert/détruit une fenêtre et terminé en code 0 ; cela a isolé le problème à l'application O93 plutôt qu'à Xvfb. Le premier smoke applicatif a bloqué dans l'initialisation ; le diagnostic précis est conservé dans les journaux.

Interprétation : la persistance d'une boucle ne suffit pas à prouver une autonomie forte. L'actionneur maintenant ajouté ne fait que des opérations locales autorisées ; les tâches sans données ne sont pas inventées, les évaluations négatives déclenchent le diagnostic, et l'apprentissage est différé en l'absence de paires explicites. Le moteur ne choisit pas arbitrairement de matériel ni de réseau.

Conséquences : théorique – séparer la boucle de contrôle, la mémoire de raisonnement et l'audit rend l'état inspectable ; conceptuelle – un retour négatif change la priorité, pas la vérité ; opérationnelle – verrou exclusif, action résultat, arrêt contrôlé et reprise par checkpoint sont mesurables.

Statut : `RUNNING / NOT YET ACCEPTED` ; l'essai GUI précédent a échoué et ne sera clos qu'après une nouvelle exécution instrumentée et une régression complète. Les essais Windows natifs et

matériels restent une gate ultérieure, non substituables par Xvfb.

B.1 P6 – rejeu après correction de l'ordonnanceur UI et ajout de l'actionneur autonome

Préflight : `PASS_BOUNDED` ; le prévol a relu les documents canoniques, la checklist et les registres d'erreurs O92/O93 ; les hashes des deux sources sont consignés par le résultat préflight.

Résultats de la suite autonome : `tests.test_p6_autonomy` termine en code 0 avec 7/7 tests réussis, rapport structuré `logs/P6_test_report.json`. En plus des critères initiaux, la boucle exécute maintenant au plus deux actions locales et auditées par pulse. En workspace vide elle empreinte un instantané interne et écrit `observations/autonomy_self_inventory.json` ; sans labels elle diffère explicitement l'apprentissage ; sur feedback négatif elle donne priorité à l'analyse, relit l'entrée si disponible et propose des sondes, sans traiter la note comme vérité. Le test prouve qu'OFF ne coupe que le journal de raisonnement : la boucle et la chaîne d'audit continuent. Arrêt, checkpoint, libération du lock, redémarrage et double instance sont contrôlés.

Défaut UI reproduit : le banc a trouvé que `_build_autonomy` démarrait une tâche de statut asynchrone avant que `_refresh_status` ne programme une seconde tâche ; l'état `busy` déclenchait une boîte modale et bloquait le constructeur Tk. Le traceur a isolé l'étape exacte. La correction retire l'appel de statut redondant et rend le handler d'arrêt explicite : si l'autonomie n'a pas confirmé `STOPPED`, l'interface reste ouverte et signale l'erreur. Le test applicatif Xvfb après cette correction termine en code 0 : la vraie fenêtre expose cinq onglets, l'autonomie démarre et produit un pulse/action, puis la commande d'arrêt arrête le runtime, écrit le checkpoint, libère le lock et conserve les deux journaux intègres.

Statut P6 : `RUNNING`. La partie runtime autonome passe 7/7, mais la phase ne ferme pas tant que le vrai constructeur/interface O93 n'a pas été lancé, qu'au moins un pulse/actions n'a pas été observé puis arrêté par l'UI, et que le checkpoint, l'intégrité du journal, le lock libéré et le code de sortie 0 ne sont pas enregistrés. La régression dépendante P0–P6 doit être rejouée après acceptation de ce smoke.

B.1.1 P6 – preuve d'acceptation de l'interface réelle

Description : vérification de la véritable classe `O93Desktop`, pas d'une maquette ou d'un simple import.

Pipeline : préflight `P6-ui-autostart-post-fix` en `PASS_BOUNDED`. Le script démarre le runtime, construit la fenêtre réelle, attend un pulse et une action, invoque le handler d'arrêt puis vérifie le runtime et le checkpoint. **Commande exacte** :

```
TERM=dumb PYTHONPATH=Dmodel_AAM_O93 timeout 25s xvfb-run -a python -u
    Dmodel_AAM_O93/scripts/p6_ui_autostart_smoke.py
```

Résultats : code de sortie 0 ; cinq onglets visibles (Chat/Tâche, Observer, Sources/RAG, Sondes/Compétences, Autonomie/Mémoire) ; démarrage `RUNNING` ; au moins un pulse et une action observés ; après arrêt `STOPPED`, runtime fermé, checkpoint présent, lock absent, intégrité audit et mémoire de raisonnement `PASS`. La première tentative UI, auparavant bloquée, reste préservée comme historique. Le diagnostic O93-E020 est résolu par la suppression du second statut asynchrone pendant la construction et par un handler qui ne masque plus un arrêt non confirmé.

Interprétation : l'interface ouvre bien la boucle autonome dans ce test Linux/Xvfb ; cela ne vaut pas acceptation Windows native. L'action autonome observée est locale et bornée ; la caméra et le réseau ne sont pas déclenchés implicitement.

Statut : test UI `PASS_BOUNDED` ; phase P6 toujours `RUNNING` en attente de la régression intégrée P0–P6 et contrôle final des artefacts.

B.2 Clôture de P6 et transition contrôlée vers P7

Description : clôture basée sur le rapport `logs/P6_complete_report.json`, sans effacer les anciennes tentatives de GUI qui ont échoué.

Commandes : la suite P6 termine à 7/7 ; le smoke de la classe réelle `O93Desktop` sous Linux/Xvfb termine en code 0 ; la régression `tests.test_p0_foundation` à `tests.test_p6_autonomy` termine à 53/53, code de sortie 0, log `logs/P0_P6_regression_final.txt`.

Résultats vérifiés : l'application expose cinq onglets ; l'autonomie démarre en `RUNNING`, produit un pulse et une action locale ; le handler UI obtient `STOPPED`, écrit `checkpoints/autonomy_stop_checkpoint.json`, supprime le verrou et laisse les deux journaux intègres. L'unité de reprise vérifie qu'un deuxième runtime ne vole pas le verrou et qu'un verrou illisible n'est pas supprimé automatiquement.

Conséquences : P6 établit une boucle persistante qui observe et exécute un petit agenda sûr local, avec arrêt/reprise mesurés. Elle ne justifie pas encore l'autonomie générale : l'apprentissage auto est différé en l'absence de labels fiables, la caméra/réseau ne sont pas auto-ouverts, et Windows natif n'a pas été testé.

Statut : P6 = `COMPLETE` dans son périmètre d'acceptation ; checkpoint signé par SHA-256 `checkpoints/P6_complete.json`. La phase P7 commence maintenant, sans préjuger de ses critères multimodaux et de génération.

Annexe C

P7 – intégration chat, génération, média et Observer

Description : tester le passage par le contrat `TaskEnvelope`, l'observation des entrées image/audio/vidéo, la route de chat sur contenu déjà observé, la génération via une capsule LM livrée et la réponse explicite lorsqu'une compétence générative de média n'existe pas.

Architecture : le runtime inspecte le goal et les entrées avant de choisir une compétence. Le chemin de création textuelle appelle la capsule d'auto-attention embarquée et marque toute sortie comme non vérifiée. Les demandes d'image/audio/vidéo sont maintenant séparées de la génération textuelle ; en l'absence d'un générateur de cette modalité, le résultat est `NOT_RUN` au lieu de produire du texte en prétendant avoir créé un média. L'observateur garde les états de décodage distincts de l'empreinte de bytes.

Pipeline/commande : préflight `P7-integrated-tests` `PASS_BOUNDED`, puis `python -m unittest tests.test_p7_integrated -v`. Le premier passage a fait 5/6 ; l'unique échec est une assertion trop littérale sur une phrase du résultat lexical. Le comportement retourné énonce en français qu'aucun classement neuronal sémantique n'est revendiqué. L'erreur de banc O93-E021 est ouverte, la sortie brute est conservée dans `logs/P7_test_integrated_initial.txt`, et la phase reste `RUNNING`.

Résultats déjà démontrés : la génération texte route sur le modèle de langage local de la capsule empaquetée et produit `GENERATED_UNGROUNDED` ; les attachements PNG et WAV reçoivent le statut de décodage `VALID` ; un faux fichier MP4 ne reçoit pas un statut de décodage valide ; le chat sur contenu observé renvoie une citation et la méthode BM25-like ; les créations de média non supportées sont signalées `NOT_RUN` sans retomber silencieusement sur le générateur texte.

Interprétation : cela valide les frontières de route et les états honnêtes, pas la qualité générale du chat ni la compréhension vidéo. Le décodeur vidéo est absent/non configuré dans le runtime actuel et le statut `NOT_DECODED` est attendu pour l'échantillon invalide. La génération image/audio/vidéo reste non implémentée tant qu'un générateur de modalité entraîné ou un provider explicitement enregistré n'existe pas.

Conséquences : théorique – l'empreinte bas niveau n'est pas assimilée à un décodage sémantique ; conceptuelle – chaque demande choisit une compétence de modalité explicite ; opérationnelle – outputs et citations gardent un statut et des hashes vérifiables.

Statut : P7 `RUNNING` ; répare l'assertion de test et rejoue P7, puis smoke UI de workflow et régression P0–P7 avant fermeture.

C.1 P7 – rejeu intégré après normalisation de l'assertion

Préflight : `PASS_BOUNDED` ; formalisme, laboratoire, checklist et registres d'erreurs consultés avant le rejeu.

Description/architecture : six tests exercent le routage par `TaskEnvelope`, le modèle de langage local chargé depuis capsule, l'observation des pièces jointes image/WAV, le refus d'une

fausse génération image/audio/vidéo par repli texte, le statut explicite `NOT_DECODED` d'une vidéo non décodable et le retrieval observation-first avec citations.

Pipeline/commande : la suite dédiée passe au second essai, journalisé avec code de retour. **Commande** :

```
TERM=dumb PYTHONPATH=Dmodel_AAM_093 timeout 90s python -m unittest tests.
    test_p7_integrated -v
```

Journal : logs/P7_test_integrated_attempt2.txt ; code retour 0.

Résultats : 6 tests, 6 réussis, 0 échec. L'essai précédent reste conservé avec 5/6 ; l'unique défaut provenait d'une assertion de texte trop littérale. Le test vérifie maintenant à la fois le champ structuré `retrieval_method` (BM25-like lexical) et la réserve réellement formulée par le runtime. Aucun comportement du modèle n'a été modifié pour satisfaire l'assertion.

Interprétation : cela vérifie les contrats bornés des voies testées ; cela ne prouve pas une génération image/audio/vidéo, une compréhension vidéo, une recherche sémantique neuronale, ni la généralisation de la petite capsule de langage.

Conséquences : théorique – un contrat structuré est plus stable qu'un fragment de phrase arbitraire ; conceptuelle – une demande hors capacité n'est pas maquillée en réponse textuelle ; opérationnelle – les états `NOT_RUN`, `NOT_DECODED` et la méthode lexicale restent visibles.

Statut : `P7 RUNNING`. Il reste à tester le workflow de la vraie interface (chat avec pièce jointe, onglet Observer, arrêt autonome et checkpoint) avant fermeture de phase.

C.2 P7 – essai réel de workflow de l'interface

Description : vérification via l'interface Tk complète, pas seulement via appel direct du runtime : lancer AAM, joindre une image, soumettre une tâche dans l'onglet Chat/Tâche, observer le même fichier dans l'onglet Observer puis arrêter l'autonomie et vérifier checkpoints/journaux.

Architecture/pipeline : `O93Desktop` démarre l'`AAMRuntime` et la boucle de fond ; les contrôles UI dirigent les tâches par le runtime commun. Le test sous Xvfb renseigne les widgets comme un opérateur, déclenche les mêmes callbacks, attend leurs résultats asynchrones et valide l'arrêt propre.

Commande :

```
TERM=dumb PYTHONPATH=Dmodel_AAM_093 timeout 40s xvfb-run -a python
    Dmodel_AAM_093/scripts/p7_ui_workflow_smoke.py
```

Sortie brute : logs/P7_ui_workflow_attempt1.txt ; code 0 ; rapport structuré : logs/P7_ui_workflow_report.json.

Résultats : fenêtre ouverte avec cinq onglets ; autonomie initiale `RUNNING` ; le chat avec image jointe retourne `PASS_BOUNDED` et `decode_status=VALID` ; l'onglet Observer retourne `PASS_BOUNDED` ; deux pulses d'autonomie sont observés avant arrêt ; le runtime est fermé, statut après stop `STOPPED`, checkpoint présent, verrou supprimé, intégrité audit et mémoire de raisonnement `PASS`.

Interprétation : la boucle et les callbacks de l'interface sont réellement exécutés sur Linux/Xvfb. Cela n'est pas une validation Windows native ni une preuve d'accès matériel réel. Le nombre d'observations retournées par le code Observer est un objet d'observation unique, pas un tableau d'observations — le rapport l'indique par ses clés et le statut du résultat.

Conséquences : théorique – les commandes UI partagent le même contrat runtime ; conceptuelle – « Observer » reçoit un fichier réel et expose l'état de décodage ; opérationnelle – le chat, la pièce jointe, l'observation et l'arrêt/checkpoint sont prouvés bout en bout.

Statut : le workflow UI est `PASS_BOUNDED`. P7 reste `RUNNING` jusqu'à test du feedback utilisateur et régression dépendante finale. Génération image/audio/vidéo, caméra physique et Windows natif restent `NOT_RUN`.

C.2.1 P7 feedback utilisateur – audit avant correction

Description : audit statique du contrat entre tâche, interface, feedback et boucle autonome. **Résultat** : le runtime fournit `rate_task(task_id, rating, comment)` et l'autonomie exploite

le dernier feedback pour reprioriser les diagnostics ; toutefois, l'interface graphique n'exposait pas encore de contrôle de notation relié au `task_id` visible. Ce constat est un manque fonctionnel de l'UI, enregistré comme `093-E022` ; aucun test d'exécution n'est déclaré dans cet audit. **Correction prévue** : afficher le task ID renvoyé, fournir les choix `satisfied/partial/unsatisfied`, persister par `rate_task`, puis tester depuis le vrai callback GUI que la note apparaît dans le journal et modifie l'agenda sans remplacer les preuves. P7 demeure ouverte jusqu'à réussite de cette preuve.

C.2.2 P7 feedback UI – implémentation et vérification de bout en bout

Correction : l'interface Chat/Tâche présente maintenant l'identifiant de la dernière tâche, trois évaluations `satisfied/partial/unsatisfied`, un commentaire et le bouton « Enregistrer note ». Le callback appelle le mécanisme canonique `runtime.rate_task` ; le runtime et le registre de feedback n'ont pas été remplacés par un chemin de test.

Commande/pipeline : le script `scripts/p7_ui_workflow_smoke.py` ouvre l'application Tk, joint une image, soumet la tâche, utilise l'onglet Observer, clique programmatiquement le callback de notation avec la valeur `unsatisfied`, vérifie l'événement d'audit et l'agenda, puis ferme AAM. Commande :

```
TERM=dumb PYTHONPATH=Dmodel_AAM_093 timeout 40s xvfb-run -a python
    Dmodel_AAM_093/scripts/p7_ui_workflow_smoke.py
```

Résultats : code 0 ; cinq onglets ; la pièce jointe image est `VALID` ; l'onglet Observer passe ; tâche de feedback `170eacc9-59bb-44d3-9713-797cf0a3c9ce` ; note `unsatisfied`, récompense `-1.0`, feedback retrouvé dans l'audit ; l'agenda place `diagnose_user_unsatisfied_result` en premier ; la politique « rating n'est pas vérité » reste présente ; 2 pulses autonomes ; checkpoint présent, lock retiré, audit et mémoire narrative intègres ; exit code 0. Rapport : `logs/P7_ui_workflow_report.json`.

Interprétation/conséquences : l'interface expose désormais le signal utilisateur demandé et l'orchestrateur peut le consommer comme priorité, sans faire du jugement utilisateur un label factuel. Prouvé sur Tk/Xvfb ; Windows natif et caméra réelle restent non exécutés.

Statut : défaut `093-E022` résolu. P7 attend encore la régression dépendante complète P0–P7 avant fermeture formelle.

C.3 Clôture de P7 et régression dépendante P0–P7

Description : après ajout du feedback UI lié au task ID et test de bout en bout, huit suites de phase sont rejouées dans le même environnement pour contrôler les dépendances modifiées.

Commande :

```
TERM=dumb PYTHONPATH=Dmodel_AAM_093 timeout 120s python -m unittest
    discover -s Dmodel_AAM_093/tests -p 'test_*.py' -v
```

Journal : logs/P0_P7_regression_attempt1.txt ; code 0 ; SHA-256 journal :

`48dd68495d0a4a6c8d3b522f0f7f29103d06a3f933d1430fc6220c501320c86`

Résultats : P0 6/6 ; P1 11/11 ; P2 9/9 ; P3 8/8 ; P4 6/6 ; P5 6/6 ; P6 7/7 ; P7 6/6 ; total **59/59**, zéro échec et sortie 0. Le smoke test GUI de P7, distinct des tests unitaires, passe aussi en code 0 et vérifie pièce jointe, onglet Observer, feedback insatisfait persistant, priorité d'agenda, checkpoint et chaînes d'intégrité. Un avertissement terminal `TERM` non configuré est survenu après le résumé de tests, sans impact sur les résultats/exit code ; il est classé environnement et les futures captures non interactives définiront `TERM=dumb`.

Interprétation : P7 a satisfait les gates du périmètre Linux/Xvfb et des fixtures. Les sorties de création audio/image/vidéo restent explicitement `NOT_RUN` ; aucune caméra physique ni Windows natif n'a été testée.

Conséquence/statut : P7 passe en `COMPLETE` / `PASS_BOUNDED`. Le checkpoint est checkpoints/P7_complete.json. P8 est la prochaine phase et ne commence qu'après enregistrement de cette fermeture et un nouveau préflight.

C.3.1 Transition P7–P8 – incident de copie de checklist, sans exécution du modèle

Résultat du préflight : la première tentative de préflight P8 a renvoyé `BLOCKED / invalid O93 work checklist schema`. L'inspection a montré que la checklist canonique racine était correcte, mais que le script de consolidation des rapports avait copié le registre d'erreurs d'environnement à sa place dans les trois miroirs embarqués, à cause d'une variable de chemin réutilisée.

Impact : aucun test P8 n'a été lancé ; le blocage a été détecté avant exécution. La source canonique n'a pas été altérée. Incident classé `O93-ENV009`, puis résolu en copiant les miroirs depuis la source canonique et en comparant les SHA-256.

Statut : P7 demeure `COMPLETE` ; P8 reste `NOT_STARTED` jusqu'au nouveau préflight. Cette erreur d'outillage est gardée dans l'historique, elle n'est pas effacée du registre.

Annexe D

P8 – sécurité, intégrité et résilience (phase ouverte)

Préconditions : P7 est `COMPLETE/PASS_BOUNDED`, étayée par 59/59 tests de régression, UI E2E, feedback persistant et checkpoint ; le préflight a passé après restauration byte-identique des miroirs. P8 est ouverte comme phase courante, sans résultat anticipé.

Plan de tests : (1) altérer volontairement une copie de journal et vérifier que la chaîne détecte l'altération ; (2) altérer un checkpoint puis vérifier son rejet ; (3) tenter observation d'un symlink direct et d'un répertoire qui en contient ; (4) dépasser volontairement les budgets fichier ; (5) fournir JSON malformé/bytes inconnus ; (6) inspecter du code sans l'exécuter ; (7) contrôler que la caméra n'est jamais ouverte par découverte et requiert autorisation ; (8) tester l'arrêt, le checkpoint, la libération du verrou et le refus d'un deuxième runtime ; (9) vérifier la présence éventuelle de code d'extraction archive et ses contrôles contre traversal ; (10) stress borné des budgets et vérification d'un état d'échec explicite.

Méthode : chaque test prépare ses données temporaires isolées, injecte une anomalie, vérifie le statut exact attendu puis confirme qu'aucune donnée source n'a été modifiée. Le résultat n'est pas accepté si les tests de régression P0–P7 ou l'intégrité des journaux/checkpoints échouent. Les cas impossibles à tester dans cet environnement sont consignés `NOT_RUN`, pas assimilés à un succès.

Incidents de préparation : `O93-ENV009` documente une erreur de variable ayant écrasé les trois miroirs de checklist ; la source canonique était intacte, le préflight a bloqué avant P8 et les miroirs ont été restaurés et hashés. `O93-ENV010` documente une faute de chemin pendant une lecture statique, sans exécution du modèle ; elle est résolue. Ces événements restent dans les registres.

Statut : `P8 RUNNING` ; prochaine action : exécuter des tests adversariaux dédiés après un nouveau préflight.

D.0.1 P8 – première suite de sécurité : défaut de symlink détecté

Commande : `PYTHONPATH=Dmodel_AAM_O93 TERM=dumb timeout 60s python -m unittest tests.test_p8_security -v`. Journal : `logs/P8_security_attempt1.txt` ; code de sortie 1.

Résultats : 11 tests, 10 réussis, 1 échoué. Ont réussi : altération de chaîne d'audit détectée ; checkpoint altéré rejeté ; ZIP traversal, entrée symlink et budgets d'expansion rejetés ; JSON à clé dupliquée/non-finite rejeté ; observateur qui refuse symlink direct et gros fichier ; code Python observé par AST sans effet de bord ; caméra exigeant autorisation ; client Internet désactivé et IP privée refusée sans requête réseau.

Défaut réel : le test `DatasetLoader.load` a prouvé qu'un chemin de dataset symlink était accepté parce que la fonction appelait `resolve()` avant `is_symlink()`. Une fois résolu, le chemin final n'était plus lui-même un symlink. L'erreur est enregistrée `O93-E023`. P8 reste ouverte ; aucune promotion de sécurité ne peut être faite avant correction du contrôle sur le chemin original et ses composants, puis rejeu.

D.0.2 P8 – script de correction partiellement appliqué, sans test post-correctif

Le premier script de patch a modifié le contrôle de chemin dans `aam_o92/data.py`, puis s'est arrêté car le bloc attendu du test ne correspondait pas textuellement au fichier réel. Le test n'a pas été modifié et aucun essai post-correctif n'a été exécuté. Incident de préparation `O93-ENV011` ouvert ; l'état du code est considéré non validé tant que le test de lien direct et de répertoire parent n'a pas été écrit, préflighté puis rejoué.

Journal de protocole – invocation ciblée invalide (résolue au niveau du banc, pas encore du test de code). Une tentative du test unitaire ciblé a référencé une classe inexistante, `P8SecurityTests`, alors que la classe déclarée est `Phase8SecurityTests`. Le processus a retourné 1 avant le test de sécurité ; ce résultat n'est pas interprété comme réussite ou échec du chargeur. L'incident est inscrit sous O93-ENV012 ; le test ciblé doit être relancé sous son identifiant exact après préflight.

Journal de maintenance – récupération des miroirs. Une commande de synchronisation a rencontré `SameFileError` en copiant un registre privé sur lui-même après avoir actualisé les fichiers canoniques et le miroir Dmodel. Aucun test de modèle n'a été exécuté par cette commande. La reprise ignore maintenant les couples source/destination identiques, synchronise Dmodel/Amodel/suivi et vérifie explicitement les empreintes SHA-256. L'incident est conservé sous O93-ENV013.

P8 – correction du lien symbolique DatasetLoader. Le test ciblé `Phase8SecurityTests.test_dataset_loader_refuses_leaf_and_parent_symlinks_before_resolve` passe 1/1 (code retour 0). Le chargeur inspecte désormais les composantes du chemin avant `Path.resolve()`, refusant à la fois un lien sur le fichier et un parent symbolique. Le défaut O93-E023 est résolu pour ces scénarios testés ; la gate P8 demeure ouverte jusqu'à la suite adversariale complète. Les erreurs du banc O93-ENV011/012 et de miroir O93-ENV013 sont résolues avec leurs traces conservées.

D.0.3 P8 – suite adversariale initiale après corrections

Description : la suite `tests.test_p8_security` couvre chemins symboliques, ZIP malveillants, corruption de journaux et checkpoints, sauvegardes de configuration, redaction, budgets, ressources, refus réseau et autorisation de capteur. **Architecture/pipeline** : préflight O93 (docs, checklist, erreurs O92/O93) puis exécution unitaire en environnement local sans accès réseau externe ; chaque résultat est conservé dans le log et le rapport JSON. **Commande** : `python -m unittest tests.test_p8_security -v`. **Résultats** : 15 tests réussis, 0 échoué, code retour 0. O93-E023 (symlinks) et O93-E024 (sauvegarde `.bak`) ont des tests correctifs réussis. **Interprétation** : les scénarios adversariaux couverts se terminent comme attendu. Cette suite est un ensemble de tests, pas une preuve d'absence de toutes vulnérabilités. **Conséquences théoriques/conceptuelles/opérationnelles** : il faut préserver le principe fail-closed et l'audit des échecs. Le correctif du chargeur influence P1/P5 et la persistance influence P0/P5 ; les gates dépendantes seront rejouées avant clôture. **Conclusion/statut** : `PASS_BOUNDED` ; P8 reste `RUNNING` jusqu'à la régression des phases dépendantes.

Audit de métadonnées du rapport P8. Le premier parseur du résumé JSON n'a pas extrait les noms individuels malgré le résumé `unittest` valide. Le parseur a été corrigé depuis le log brut, avec 15 noms explicitement associés aux 15 tests réussis. Les assertions n'ont pas été modifiées ni rejouées par cette correction ; O93-ENV014 est résolu au niveau du rapport.

Journal de commande – préflight importé du mauvais module. Une première commande de régression s'est arrêtée avant test car elle référençait `aam_o92.preflight` ; la fonction O93 se trouve dans `aam_o93.preflight`. Aucun résultat de test n'est associé à cet appel. L'incident environnement O93-ENV015 reste à résoudre par un préflight correct et un rejeu P0–P7.

D.0.4 P8 – rejeu des phases dépendantes

Description : après correction du parcours symbolique dans `DatasetLoader` et maintien de la sauvegarde `.bak`, la suite des phases déjà achevées a été rejouée pour détecter toute régression. **Architecture** : les tests P0 à P7 restent les mêmes gates ; seuls l'environnement, le chargeur et la persistance sont mis à jour. **Pipeline/commande** : préflight O93 lisant les documents, checklists et registres, puis la suite P0–P7 :

```
TERM=dumb PYTHONPATH=. python -m unittest tests.test_p0_foundation tests.
   test_p1_observer tests.test_p2_probes tests.test_p3_competencies tests.
   test_p4_acquisition tests.test_p5_knowledge tests.test_p6_autonomy
   tests.test_p7_integrated -v
```

Résultats : 59 tests réussis, 0 échoué, code retour 0. **Interprétation** : les tests couverts n'ont pas révélé de régression dans l'observation, les sondes, les compétences, l'acquisition, l'ingestion observation-first, l'autonomie ou le routage du chat. **Conséquences** : les changements de sécurité restent compatibles avec les contrats des phases dépendantes, sous les domaines et données testés. **Conclusion/statut** : `PASS_BOUNDED` ; la fermeture de P8 attend encore le smoke UI de régression et l'audit final des artefacts.

D.0.5 P8 – synthèse de sortie avant clôture

Description : validation adversariale, retour arrière, confidentialité, permissions, corruption, ressources et absence d'exécution des contenus observés. **Architecture/pipeline** : préflight normatif ; 15 tests de sécurité ; rejeu des phases P0–P7 ; smoke test réel Tk/Xvfb ; consolidation des journaux et des empreintes des copies. **Commandes d'exécution** (depuis `Dmodel_AAM_O93`) :

```
TERM=dumb PYTHONPATH=. python -m unittest tests.test_p8_security -v
TERM=dumb PYTHONPATH=. python -m unittest tests.test_p0_foundation tests.
   test_p1_observer tests.test_p2_probes tests.test_p3_competencies tests.
   test_p4_acquisition tests.test_p5_knowledge tests.test_p6_autonomy
   tests.test_p7_integrated -v
timeout 45s xvfb-run -a python scripts/p7_ui_workflow_smoke.py
```

Résultats : sécurité 15/15 ; régression P0–P7 59/59 ; interface 5 onglets, pièce jointe image `VALID`, feedback lié au `task_id`, 2 pulses d'autonomie, checkpoint conservé, verrou libéré, deux journaux intègres ; code retour UI 0. **Interprétation** : les protections et scénarios d'intégration testés répondent aux contrats déclarés. Ils ne certifient pas l'absence universelle de vulnérabilités. **Conséquences théoriques/conceptuelles/opérationnelles** : le modèle conserve une politique fail-closed et des preuves rejouables ; les entrées matérielles réelles et Windows natif restent des gates distinctes. **Conclusion/statut** : rapport de sortie `P8_gate_exit_report.json` indique `PASS_BOUNDED_EXIT_CRITERIA_MET` ; décision de fermeture formelle inscrite dans la checklist après examen des erreurs ouvertes liées à P8.

Correction de la nomenclature des erreurs P8. Le défaut de sauvegarde de configuration correspond à O93-E026 ; le défaut de redaction des champs sensibles correspond à O93-E025. Le rapport de sortie a été corrigé pour ne pas attribuer ces preuves à O93-E024. Les entrées historiques demeurent conservées ; seule la référence du rapport de synthèse est normalisée.

Décision P8 – critères locaux atteints. Les preuves réunies sont : 15/15 tests adversariaux P8 (code retour 0), 59/59 tests dépendants P0–P7 après correction du chargeur et de la persistance (code retour 0), puis le workflow réel Tk/Xvfb (code retour 0, 5 onglets, pièce jointe observée, feedback tracé, checkpoint/arrêt intègres). Les erreurs spécifiques O93-E023/E025/E026 et ENV011–ENV015 sont résolues par des preuves. La fermeture concerne seulement le périmètre local testé ; les items d'architecture ouverte (E001–E005), Windows natif (ENV001) et ressources/matériel (ENV002/003) demeurent suivis dans P9/P10.

D.1 Historique des protocoles P9 antérieurs et tentative stricte non concluante

D.1.1 Tentative 01 – cinq graines et worker source-off : échec de mesure

Description : une première tentative a entraîné et sauvegardé cinq capsules FPN pour les graines 13, 29, 47, 71 et 101, mais n'a pas produit de mesures source-off acceptables. L'entraînement de ces capsules n'est pas interprété comme une réussite de la gate de capacité, car l'étape de replay/évaluation n'a pas abouti.

Architecture et pipeline : copie de TRAIN dans un espace temporaire, acquisition et export de capsule, retrait de la copie d'entraînement, création d'un bundle minimal, demande d'exécution du worker source-off, puis assemblage du rapport. L'assembleur a échoué après la demande d'inférence, en essayant de relire le script worker après suppression du répertoire temporaire et d'écrire dans un flux d'événements déjà fermé.

Commande tentée :

```
timeout 180s python scripts/p9_capacity_evaluation.py --protocol protocols
   /P9_preregistered_capacity_protocol.json
```

Résultats : code retour 1 ; les cinq fichiers de run portent le statut `ERROR` ; aucun score source-off n'a été archivé. L'analyse est conservée sous `logs/P9_attempt01_failure_analysis.json` ; les capsules restent sous `trained_artifacts/p9_capacity_eval_preregistered01`. Aucun chiffre de cette tentative n'est utilisé dans les métriques P9 actuelles.

Interprétation : l'incident est un défaut du banc de mesure et de son cycle de vie des ressources. Il ne démontre ni succès ni échec de la compétence, puisque les métriques d'inférence n'ont pas été obtenues. Il reste important pour la traçabilité car l'entraînement a bien eu lieu, contrairement aux premières erreurs de préflight qui s'arrêtaient avant le fit.

Conséquences théoriques, conceptuelles et opérationnelles : la présence d'une capsule entraînée ne vaut pas preuve de replay indépendant ; le worker doit rester disponible jusqu'à la fin du reporting et les flux de journaux doivent avoir un propriétaire unique. Le statut d'acceptation du chemin strict reste ouvert tant qu'un worker isolé ne termine pas avec des métriques valides.

Conclusion/statut : tentative 01 `FAIL_REVIEW_REQUIRED` ; aucune mesure source-off valide, aucune compétence promue. L'incident modèle `O93-E027` reste `OPEN` tant que la procédure stricte n'a pas été corrigée et rejouée.

D.1.2 Historique de préinscription et protocole mesuré ensuite

Une proposition antérieure prévoyait cinq graines (13, 29, 47, 71, 101), 60 époques, un budget de sondes de 1,2 seconde et deux rounds, avec un worker séparé sans accès au répertoire de dataset. La validation de préinscription et la compilation statique sont des preuves de préparation, pas des résultats de capacité. La tentative à cinq graines décrite ci-dessus a bien lancé l'entraînement, mais son évaluation stricte a échoué et ne fournit aucune métrique valide.

Le run exploratoire V1 décrit ensuite a utilisé les graines 93, 193 et 293, 40 époques, un budget de 2 secondes et 3 rounds. Il a vérifié les hashes avant entraînement mais n'a pas utilisé un processus enfant privé du dataset ; le prédicteur recevait le seul champ `input` tandis que le dataset restait accessible dans le même environnement. Cette tentative historique n'est pas une preuve d'isolation stricte source-off. Le protocole V2 à cinq graines, présenté plus loin et réussi avec workers séparés, remplace V1 comme expérience principale de capacité.

Traces et statut V1 : `logs/P9_preregistered_protocol_attempt1.json`, `logs/P9_static_compile_attempt2.txt`, `logs/P9_preregistration_validation.json` et `logs/P9_attempt01_failure_analysis.json`. V1 est conservée comme expérience exploratoire. Les mesures V1 suivantes sont historiques ; la section V2 ultérieure est la référence finale et apporte le worker séparé source-off.

Annexe E

P9 – essai exploratoire V1 à trois graines (historique ; remplacé par V2)

E.1 Description

P9 évalue la stabilité de l'acquisition de compétence texte-forme et le transfert après rechargement des capsules. Le prédicteur reçoit seulement le champ input ; le dataset reste présent dans le même environnement d'évaluation, donc ce run ne prouve pas une isolation stricte par processus. Le protocole a été préenregistré avant l'entraînement : graines 93, 193 et 293 ; 40 époques ; budget de recherche de sondes de 2 secondes et 3 rounds par graine ; seuil d'abstention de 0,55 ; mêmes entrées, partitions, métriques et règle de non-promotion pour les trois runs. Le protocole initial dont le garde-fou de schéma a échoué est préservé sous `logs/P9_preregistered_protocol_attempt1.json` ; l'amendement suivant n'a changé ni données, ni graines, ni paramètres, ni métriques, seulement ajouté un alias du hash de script attendu.

E.2 Architecture

Sondes : `ProbeFactory` génère et évalue les sondes sur les seules entrées TRAIN.

Composition : `O93AcquisitionService` compose les sorties choisies avec l'empreinte de base.

Apprentissage et capsule : `FingerprintNeuronNetwork` apprend les poids avec Adam et régularisation L2 ; la capsule sauvegarde les poids, le vectoriseur, les définitions de sondes et la provenance, puis est rechargée pour l'inférence.

Séparation des données : le jeu de défi `examples/datasets/o93_shape_transfer_challenge.jsonl` n'est ni passé à l'acquisition ni fourni au sélecteur de sondes. Le processus d'évaluation conservait toutefois un accès au système de fichiers ; les cibles sont jointes après chaque prédiction. Il ne s'agit pas d'un worker séparé privé du dataset.

E.3 Pipeline complet

Préflight formalisme/laboratoire/checklists/registres → vérification du hash du protocole, du script et des deux datasets → audit des partitions et des entrées normalisées → trois acquisitions indépendantes par graine → enregistrement des poids et des capsules → rechargement de chaque capsule → prédiction API à partir du seul champ `input` sur VALID/HOLDOUT/OOD et TRANSFER_TEST → jonction postérieure des labels → calcul de précision, précision équilibrée, couverture, abstentions, confiance, coût et erreurs → tests de cohérence des preuves → régression P0–P9.

E.4 Commandes d'exécution

Depuis `Dmodel_AAM_O93` :

```
TERM=dumb PYTHONPATH=. python scripts/run_p9_capacity.py
TERM=dumb PYTHONPATH=. python -m unittest tests.test_p9_capacity_evidence
    -v
TERM=dumb PYTHONPATH=. python -m unittest tests.test_p0_foundation tests.
    test_p1_observer tests.test_p2_probes tests.test_p3_competencies tests.
    test_p4_acquisition tests.test_p5_knowledge tests.test_p6_autonomy
    tests.test_p7_integrated tests.test_p8_security tests.
    test_p9_capacity_evidence -v
```

Le préflight O93 a été exécuté avant le run et avant la suite de régression. Les hashes normatifs courants sont conservés dans la checklist et dans le checkpoint.

E.5 Données, graines et intégrité

Le dataset principal est examples/datasets/o92_demo_shapes.jsonl (14 lignes : 8 TRAIN, 2 VALID, 2 HOLDOUT et 2 OOD). Son SHA-256 est :

`d6c75b2b06f566ee9afbbb7a8f902c04e87a2bad09d25a0e13a63ecd92cf36a4`

Le défi gelé est examples/datasets/o93_shape_transfer_challenge.jsonl (16 lignes TRANSFER_TEST : 8 cercles et 8 carrés). Son SHA-256 est :

`bd9740bba93d9b48b789547be4a19adc7d2bc1dd183f094206e2216761c285b`

Les deux datasets et le script scripts/run_p9_capacity.py ont été vérifiés contre le protocole logs/P9_preregistered_protocol.json avant entraînement. L'audit n'a trouvé aucun chevauchement d'entrée après normalisation exacte entre TRAIN, VALID, HOLDOUT, OOD ou le défi de transfert ; cela ne prouve pas l'absence de chevauchement sémantique. Le dataset et ses labels restaient présents dans l'environnement d'évaluation ; seule leur transmission à la fonction de prédiction a été limitée au champ `input`.

E.6 Résultats bruts

Les trois runs ont chacun entraîné 40 époques et appris 7 742 poids. Les pertes de validation minimales étaient respectivement $8,6201 \times 10^{-5}$, $7,2288 \times 10^{-5}$ et $6,8056 \times 10^{-5}$. Les durées bout-en-bout étaient 4,13 s, 4,69 s et 4,27 s. Les mêmes quatre sondes ont été sélectionnées dans les trois runs : `text.word_frequency`, `text.word_ngrams.n1`, `text.char_ngrams.n3` et `text.normalize.casefold`. La recherche renvoie `CANDIDATE_FOUND_NOT_GENERALIZATION_PROOF` dans chaque cas.

Graine	VALID	HOLDOUT	OOD	Transfert correct	Couverture transfert
93	2/2	2/2	2/2	15/16 (93,75%)	16/16 (100%)
193	2/2	2/2	2/2	14/16 (87,50%)	15/16 (93,75%)
293	2/2	2/2	2/2	9/16 (56,25%)	16/16 (100%)

Sur les 48 prédictions de transfert agrégées, 38 sont correctes (79,17%) ; 47/48 sorties ne s'abstiennent pas (couverture 97,92%), et 38/47 sorties non-abstention sont correctes (80,85%). Neuf erreurs de classification ont une confiance déclarée au moins égale à 0,90 ; le run de graine 293 en concentre sept, avec des scores parfois proches de 1,00. Ce comportement indique une calibration faible sur le transfert. Les capsules ont des empreintes vérifiées et n'embarquent ni lignes d'entraînement brutes ni oracle. Le modèle reçoit seulement le champ `input` lors de l'appel de prédiction ; en revanche, le run n'a pas isolé cet appel dans un processus auquel le dossier de dataset aurait été retiré. Les neuf tests P9 d'intégrité des preuves passent, et la régression combinée P0–P9 passe 83/83 tests avec code retour 0.

E.7 Interprétation des résultats

Les résultats de 2/2 sur VALID, HOLDOUT et OOD ne doivent pas être surinterprétés : chaque partition ne contient que deux exemples. Le jeu de transfert fixe et équilibré est plus informatif, et il expose une variabilité importante entre graines malgré une perte de validation faible. Le réseau peut mémoriser la séparation lexicale étroite tout en commettant des erreurs de transfert avec forte confiance. La recherche de sondes produit des caractéristiques utilisables par les poids, mais son statut explicite n'affirme pas la généralisation. Les compétences demeurent `CANDIDATE_PENDING_INDEPENDENT_ACCEPTANCE` ; aucune n'est promue à partir de la seule précision obtenue.

E.8 Conséquences théoriques

La réussite d'une acquisition sur les partitions internes et la capacité de transfert sont des observables distincts. La fonction objectif réellement testée est celle du classifieur de compétence (perte de tâche avec Adam/L2 et sélection de validation) ; P9 ne vérifie pas une optimisation différentiable de tous les termes de l'objectif global d'AAM, ni une maîtrise universelle de la compétence.

E.9 Conséquences conceptuelles

Une compétence O93 doit contenir le contrat des sondes, l'empreinte, les poids et la provenance, puis être éprouvée sans accès à la source. Le run présent valide l'appel API à entrée seule après rechargement, mais ne prouve pas l'isolation stricte du processus. Toutefois, « apprendre une compétence » n'est pas équivalent à garantir une reproduction parfaite ou un transfert général. Les erreurs de confiance élevée restent des preuves à traiter, pas à supprimer du rapport.

E.10 Conséquences opérationnelles

Les ressources de ce test sont modestes : trois acquisitions de moins de cinq secondes chacune, un petit réseau et un corpus synthétique de 8 entrées TRAIN. Le budget de sondes a été fixé à 2 secondes et 3 rounds par run ; il s'agit d'une limite du protocole d'évaluation, non d'une conclusion sur le potentiel maximal du moteur. Des expériences plus larges, davantage de données indépendantes, une calibration de confiance et de nouveaux domaines sont nécessaires pour mesurer une évolution du comportement. Aucune donnée du défi de transfert n'a servi au réglage de cette expérience.

E.11 Erreurs, limites et preuves

Les incidents de préparation P9 ENV016–ENV020 sont des erreurs de garde ou de synchronisation détectées avant l'entraînement ; chaque tentative est journalisée, aucune n'a produit un résultat de modèle. Le protocole et sa première version sont conservés. Le prédicteur a reçu uniquement le champ `input`, mais l'évaluateur conservait accès au dataset dans le même environnement ; l'isolement par worker sans accès au corpus reste `NOT_RUN`. Les logs détaillés, les capsules et les prédictions par entrée se trouvent dans `logs/P9_capacity_experiment.json`, `logs/P9_capacity_experiment.txt`, `runs/P9_capacity/` et `checkpoints/P9_experiment_complete.json`. L'acceptation Windows native, les capteurs physiques et les fournisseurs externes restent hors de cette phase.

E.12 Conclusion et statut

P9 obtient `PASS_BOUNDED` pour l'expérience bornée exécutée : hashes vérifiés, absence de chevauchement exact normalisé, prédiction API à entrée seule après rechargement, 9/9 tests de cohérence

et 83/83 tests sur P0–P9. L'isolation par worker source-off reste `NOT_RUN`. Le transfert après rechargement varie de 56,25% à 93,75% et contient neuf erreurs à confiance élevée ; la compétence n'est pas certifiée comme générale et reste candidate. La prochaine phase P10 porte sur l'acceptation Windows native, l'installation et les fournisseurs/matériels externes. Le modèle global O93 ne doit pas être marqué intégralement accepté avant cette gate et les essais qui restent non exécutés.

Rapport structuré : `logs/P9_capacity_experiment.json`. **Protocole préenregistré** : `logs/P9_preregistered_protocol.json`. **Tests de cohérence** : `logs/P9_evidence_tests.txt`. **Régression complète** : `logs/P0_P9_full_regression.txt`.

Annexe F

Rapport consolidé de clôture des phases P8–P9

Description : cette phase teste des entrées adversariales, les garanties de persistance, les frontières de permission et les limites de ressources après les correctifs. Les échecs initiaux sont gardés dans les journaux ; les états ci-dessous décrivent le rejeu final et ne remplacent pas l'historique.

Architecture : (1) `O93Memory` masque les valeurs de champs sensibles et plusieurs formes courantes de secrets en ligne avant l'écriture à la fois dans l'audit et dans la mémoire de raisonnement ; (2) `AAMRuntime._write_json` écrit sur un fichier temporaire, conserve la version précédente dans `.bak`, refuse une cible symlink et remplace atomiquement ; (3) l'observateur et le chargeur de dataset imposent des budgets et refusent les symlinks selon leur politique ; (4) le lecteur JSON strict et l'extracteur ZIP rejettent les clés dupliquées, valeurs non finies, traversées, symlinks archive et limites d'expansion ; (5) le client HTTP refuse l'accès réseau désactivé et les cibles privées avant la requête.

Pipeline : préflight (formalisme, laboratoire, checklist et registres) → suite de sécurité ciblée → résolution des constats → rejeu séquentiel P0–P7 après modification du stockage partagé → nouvelle suite P8 → régression combinée P0–P8 → checkpoint de phase et vérification des chaînes d'événements.

Commandes exactes depuis `Dmodel_AAM_O93` :

```
TERM=dumb PYTHONPATH=. python -m unittest tests.test_p8_security -v
TERM=dumb PYTHONPATH=. python -m unittest tests.test_p0_foundation tests.
   test_p1_observer tests.test_p2_probes tests.test_p3_competencies tests.
   test_p4_acquisition tests.test_p5_knowledge tests.test_p6_autonomy
   tests.test_p7_integrated -v
TERM=dumb PYTHONPATH=. python -m unittest tests.test_p0_foundation tests.
   test_p1_observer tests.test_p2_probes tests.test_p3_competencies tests.
   test_p4_acquisition tests.test_p5_knowledge tests.test_p6_autonomy
   tests.test_p7_integrated tests.test_p8_security -v
```

Résultats : la suite focalisée finale passe **15/15 tests** (code retour 0) ; la régression complète P0–P8 passe **74/74 tests** (code retour 0, journal `logs/P0_P8_full_regression.txt`). Les tests couvrent la redaction dans les deux journaux ; la détection d'une altération de la chaîne ; le rejet d'un checkpoint altéré ; la sauvegarde de versions JSON précédentes ; la gestion des symlinks directs, de répertoire parent et de fichiers trop grands ; les limites de comptage et d'expansion ZIP ; les clés JSON dupliquées et valeurs `NaN` ; les contenus d'instructions non fiables ; le refus de requêtes réseau désactivées/privées ; le consentement de capteur ; et le marquage d'un binaire inconnu comme `NOT_DECODED`.

Interprétation : les deux défauts de sécurité reproduits lors du premier essai sont corrigés pour les cas testés. Les identifiants d'erreur sont normalisés : `O93-E023` demeure le défaut historique de symlink du chargeur (résolu) ; `O93-E025` documente la redaction des secrets (résolu) ; `O93-E026` documente l'absence initiale de sauvegarde précédente (résolu). Les erreurs de protocole `O93-ENV011`

et O93-ENV012 sont résolues après exécution de la suite de tests correctement nommée. Aucun échec antérieur n'a été supprimé.

Conséquences théoriques : l'intégrité d'un journal et la confidentialité de son contenu sont deux garanties distinctes ; la chaîne de hash ne remplace pas la redaction. La sauvegarde préalable complète l'écriture atomique pour un rollback humain, sans constituer une preuve de résistance à toute panne électrique.

Conséquences conceptuelles : un contenu de document qui ressemble à une instruction reste une donnée non fiable ; un outil local ne doit pas l'exécuter. Les statuts de décodage et les limites d'observation restent explicites. La redaction par motifs réduit certaines fuites mais n'est pas un détecteur universel de secrets.

Conséquences opérationnelles : tous les tests disponibles sous Linux/Python de ce run sont verts ; la documentation, les registres et les checkpoints ont été mis à jour. Windows natif, caméra physique, fournisseurs Internet réels, comportement NTFS/ACL et scénarios de coupure électrique restent NOT_RUN et ne sont pas déclarés acceptés par extrapolation.

Conclusion/statut : P8 est COMPLETE / PASS_BOUNDED pour les tests adversariaux locaux listés, avec 15/15 tests ciblés et 74/74 tests de régression. Cela n'est pas une certification de sécurité universelle. La phase P9 a depuis été exécutée sur protocole pré-enregistré ; les résultats détaillés se trouvent au chapitre P9 précédent. Le transfert après rechargement atteint 15/16, 14/16 et 9/16 exemples selon les trois graines ; les neuf erreurs de classification au seuil de confiance >= 0,90 sont conservées explicitement. Les compétences restent candidates et ne sont pas promues. La prochaine phase est P10 : acceptation native Windows, fournisseurs externes configurés et essais matériels/récupération qui restent non exécutés.

Rapports JSON : logs/P8_security_report_final.json ; logs/P9_capacity_experiment.json.

Artefacts P8 : runs/P8_security/run.json ; runs/P8_security/metrics.json ; runs/P8_security/events.jsonl ; runs/P8_security/errors.jsonl ; runs/P8_security/checkpoint.json.

Artefacts P9 : logs/P9_preregistered_protocol.json ; logs/P9_capacity_experiment.txt ; logs/P0_P9_full_regression.txt ; checkpoints/P9_experiment_complete.json.

Annexe G

P9 — audit post-entraînement et correction du vérificateur (état intermédiaire)

Description : après l'entraînement préenregistré V2, le runner a terminé avec code retour 0, écrit cinq capsules et cinq rapports source-off, puis inscrit `P9_RUN_END`. Le premier audit d'intégrité post-run a retourné `FAIL_REVIEW_REQUIRED` avec 12/13 contrôles. L'unique assertion échouée comparait les noms de chemins triés lexicalement (`seed_101` apparaît avant `seed_13`) à une liste de graines triée numériquement. Cette assertion est un défaut du vérificateur, pas une erreur d'entraînement ; son journal original est conservé sans modification.

Architecture : agrégat figé ; puis vérification en lecture seule du protocole, runner, worker et dataset par SHA-256 ; ensuite ordonnancement numérique des cinq graines ; enfin contrôle des capsules, workers, requêtes entrée-seule, partitions, abstention, journal final, erreurs, doublons et règle de non-promotion.

Pipeline exact : préflight O93 (formalisme, laboratoire, checklist et registres O92/O93) ; ouverture de l'agrégat V2 et du protocole ; vérification des hash et résultats persistés ; reconstruction de l'ordre

$$13, 29, 47, 71, 101$$

comme entiers ; contrôle de chacun des cinq workers source-off et des métriques ; production d'un nouveau rapport sans réentraîner.

Commande : depuis le dossier `Dmodel_AAM_O93` :

```
PYTHONPATH=. python scripts/verify_p9_postrun.py
```

Le vérificateur lui-même appelle le préflight avant lecture des résultats. Le résultat de cette première tentative est conservé dans `logs/P9_preregistered02_postrun_integrity.json` ; le correctif de tri a été évalué dans un second rapport distinct.

Résultat de l'audit initial : le run modèle V2 est `PASS_BOUNDED` ; cinq graines présentes et individuellement réussies, cinq workers avec code retour zéro, dataset absent des bundles de worker, champs d'entrée limités à `id,text,threshold`, labels et tags de partition non transmis, capsules sans phrases d'entraînement exactes, métriques VALID/HOLDOUT/OOD présentes ($n = 2$ chacune), événement de fin propre, journal d'erreur vide, doublon exact inter-partitions absent et near-duplicate enregistré. Le seul test refusé est l'ordre de la liste des graines, en raison du tri lexical du vérificateur.

Interprétation : l'échec ne doit pas être masqué : il démontre que la preuve de sortie du vérificateur était insuffisante. L'agrégat ne sera considéré comme audité qu'après un nouveau contrôle dont la comparaison ordonne les graines numériquement. Les métriques des modèles restent exactement celles du run V2 ; aucun modèle n'est réentraîné et aucune donnée de résultat n'est réécrite pour faire passer le contrôle.

Conséquences théoriques : la validation du protocole/du modèle et la validité du vérificateur de preuves sont deux obligations séparées. Une erreur dans l'une ne doit pas être transformée en verdict sur l'autre.

Conséquences opérationnelles : conserver le JSON en échec, créer un vérificateur versionné avec tri numérique et générer un second rapport. Ajouter l'erreur `O93-ENV030` au registre d'environnement et la résoudre uniquement après passage du rejeu.

Conclusion et statut : le run d'entraînement V2 demeure `PASS_BOUNDED` ; son audit complet est en attente de résolution du faux négatif ENV030. La compétence reste `CANDIDATE_PENDING_INDEPENDENT_ACCEPTANCE` ; les petites partitions et le near-duplicate empêchent toute promotion.

Annexe H

P9 — résultat final du protocole préenregistré V2

Description : le runner V2 a terminé avec code retour zéro et a produit cinq capsules de compétence à partir du même petit jeu fixé avant exécution. Le vérificateur post-run révisé a réussi **14/14 contrôles** en triant les graines numériquement. L'ancien rapport à 12/13 est conservé comme faux négatif du vérificateur ; il n'est pas écrasé. Aucun réentraînement n'a été effectué lors de l'audit corrigé.

Architecture : moteur de sondes textuelles (pool observé de 31 candidats sur TRAIN) ; extracteur de caractéristiques d'empreintes ; FPN classifieur entraîné (Adam, L2, early-stopping) ; capsule JSON signée par hash ; processus enfant Python neuf avec code d'inférence et capsule seulement ; requêtes d'entrée seule ; jonction des labels après prédiction dans l'évaluateur ; métriques et audit d'intégrité indépendants.

Pipeline : préflight formalisme/laboratoire/checklist/registres ; vérification de la préinscription V2 et des SHA-256 ; entraînement des graines 13, 29, 47, 71, 101 ; export et rechargement des capsules ; suppression du workspace et absence du corpus dans le bundle source-off ; lancement des cinq workers ; scoring séparé VALID/HOLDOUT/OOD ; vérification de l'abstention, des doublons et des journaux ; audit post-run numériquement trié (14/14).

Commandes exactes depuis `Dmodel_AAM_093` :

```
timeout 180s python scripts/p9_capacity_evaluation.py --protocol protocols
   /P9_preregistered_capacity_protocol_v2.json
PYTHONPATH=. python scripts/verify_p9_postrun.py
PYTHONPATH=. python -m unittest tests.test_p9_capacity_evidence -v
```

La dernière commande appartient à la régression historique P9 ; un test dédié de cohérence V2 est ajouté pour verrouiller les artefacts V2. Le run de capacité V2 a code retour 0 ; l'audit post-run V2 a aussi code retour 0.

Résultats : le corpus contient 14 lignes (8 TRAIN, 2 VALID, 2 HOLDOUT, 2 OOD), réparties en 4 exemples de cercle et 4 de carré dans TRAIN. Les graines 13, 29, 47, 71 et 101 ont toutes le statut `PASS_BOUNDED`. Le réseau a 7 742 poids, entraînement 60 époques avec Adam et L2, et capsules de 323 153 à 324 524 octets. Les durées totales par graine vont de 5,57 à 5,99 secondes ; l'inférence source-off de 1,50 à 1,69 seconde. Le bundle d'inférence contient 51 fichiers, aucun dataset, et l'API de prédiction reçoit uniquement `id,text,threshold` ; aucune cible, partition ou chemin de dataset n'est transmis. Toutes les capsules se vérifient par hash et le rapport n'y signale aucune phrase exacte d'entraînement. Le journal finit par `P9_RUN_END`, avec cinq graines terminées et zéro erreur.

Graine	VALID	HOLDOUT	OOD	Temps total (s)
13	2/2	2/2	2/2	5,57
29	2/2	2/2	2/2	5,59
47	2/2	2/2	2/2	5,75
71	2/2	2/2	2/2	5,65
101	2/2	1/2	2/2	5,99

Sur cinq graines, VALID vaut 1,00 (écart-type population 0) ; HOLDOUT vaut 1,00, 1,00, 1,00, 1,00 et 0,50, soit moyenne 0,90 et écart-type 0,20 ; l'OOD déclaré vaut 1,00 sur toutes les graines. Les partitions d'évaluation n'ont chacune que deux exemples et la majorité simple donne 0,50 d'exactitude et d'exactitude équilibrée sur ces partitions équilibrées. Le test d'abstention forcée au seuil $0,999999$ a produit une abstention sur les cinq graines ; il valide le chemin technique d'abstention, pas sa calibration utile. Les métriques avec rappel par classe et matrices de confusion sont dans `runs/P9_capacity/preregistered02_aggregate.json` et le rapport détaillé `runs/P9_capacity/preregistered02_final_report.json`.

Interprétation : le run V2 démontre, sur ce corpus miniature, un entraînement reproductible, le stockage des poids dans une capsule, une nouvelle inférence en processus séparé sans accès au corpus et un mécanisme d'abstention effectif. Il ne démontre pas une compétence générale. Un quasi-doublon cross-partition TRAIN/VALID (`row_005`/`row_009`, Jaccard tokens 0,80, même cible) est explicitement conservé comme avertissement. La meilleure sonde de développement est `text.word_frequency`, score candidat environ 0,993, séparation utile 1,00, mais son indicateur de faux découpage intra-cible vaut 1,00. Le statut reste `CANDIDATE_FOUND_NOT_GENERALIZATION_PROOF`. La compétence n'est pas promue.

Conséquences théoriques : apprendre des poids et prédire sans oracle est une étape distincte de la généralisation ; une exactitude de 100% sur deux cas ne permet pas d'estimer finement une probabilité d'erreur. Les performances doivent être lues conjointement avec l'incertitude, la couverture, les erreurs par classe, les fuites sémantiques possibles et le coût.

Conséquences conceptuelles : l'acquisition O93 a désormais une preuve concrète source-off sur un petit domaine. La génération de sondes donne des candidats utilisables par l'apprentissage, mais la métrique de faux découpage impose un travail de conception/validation supplémentaire. L'objectif d'apprendre des moteurs/algorithmes et de les composer librement n'est pas validé par cette classification à deux classes.

Conséquences opérationnelles : les prochains essais nécessitent au minimum 30 exemples indépendants par classe en HOLDOUT et 30 par classe dans une distribution réellement déplacée, annoncés dans un autre protocole avant fit. Il faut aussi travailler les seuils d'abstention, tester d'autres modalités et maintenir les erreurs de transfert/faux découpage visibles. Le run a été CPU-only et sans accès Internet, aucun corpus externe n'a été importé.

Conclusion et statut : P9 est `PASS_BOUNDED` pour le protocole V2 : cinq graines et cinq workers source-off réussis ; post-run 14/14 ; le faux négatif du vérificateur initial est résolu et conservé dans l'historique. La compétence reste `CANDIDATE_PENDING_INDEPENDENT_ACCEPTANCE` ; aucune promotion n'est autorisée sur cette expérience. P10 (acceptation Windows native, caméra réelle, fournisseurs externes, comportement NTFS/ACL et tests de coupure) reste à exécuter dans un environnement approprié.

Artefacts principaux : `logs/P9_preregistered02_postrun_integrity_retry.json` ; `runs/P9_capacity/preregistered02_aggregate.json` ; `runs/P9_capacity/preregistered02_final_report.json` ; `logs/P9_capacity_preregistered02_events.jsonl` ; `logs/P9_capacity_preregistered02_errors.jsonl`.

H.1 Rejeu du vérificateur post-run et tests unitaires V2

Description/architecture : le vérificateur lit l'agrégat V2 sans réentraîner. Il exécute d'abord le préflight O93, vérifie les hash de la préinscription, du runner, du worker et du dataset, trie les

enregistrements par la valeur entière de `seed`, vérifie les workers/capsules/partitions et le journal de fin, puis émet un JSON indépendant. Un module de tests dédié verrouille ces invariants.

Commandes : depuis `Dmodel_AAM_O93` :

```
PYTHONPATH=. python scripts/verify_p9_postrun.py
PYTHONPATH=. python -m unittest \
  tests.test_p9_preregistered_v2_integrity -v
```

Résultats : audit read-only `PASS_BOUNDED`, 14/14 contrôles, code retour 0 ; tests V2, 11/11, code retour 0. L'ordre des graines est exactement $[13, 29, 47, 71, 101]$. Les rapports sont `logs/P9_preregistered02_postrun_integrity_retry.json` et `logs/P9_v2_integrity_unittest.txt`. Aucun modèle n'a été entraîné à cette étape.

Interprétation : le rapport antérieur 12/13 reste inchangé et explique l'erreur du vérificateur initial. La nouvelle preuve passe sans retoucher les résultats V2, ce qui confirme qu'il s'agissait bien d'un faux négatif dû à l'ordre lexical. Les critères statistiques ne sont pas assouplis.

Conséquences théoriques/conceptuelles/opérationnelles : séparer la reproductibilité du modèle de la reproductibilité de l'audit est essentiel ; les outils de vérification sont eux-mêmes versionnés et testés. Toute future version du vérificateur doit garder le même ordre numérique préenregistré et les preuves d'origine.

Conclusion/statut à ce stade intermédiaire : le défaut `O93-ENV030` est résolu après rejeu. La régression combinée finale et l'inspection documentaire restaient alors ouvertes ; elles sont consignées dans les sections qui suivent.

H.2 Régression combinée finale P0–P9

Description : après résolution du faux négatif du vérificateur et ajout des tests V2, la régression combinée des phases P0–P9 a été exécutée dans l'ordre. Elle comprend les tests de fondation, observation, sondes, composition de compétences, acquisition, knowledge/observe-first, autonomie, interface intégrée, sécurité, preuves P9 historiques et preuves P9 préenregistrées V2.

Commande exacte : depuis `Dmodel_AAM_O93` :

```
timeout 180s env TERM=dumb PYTHONPATH=. python -m unittest \
  tests.test_p0_foundation tests.test_p1_observer tests.test_p2_probes \
  tests.test_p3_competencies tests.test_p4_acquisition tests.
     test_p5_knowledge \
  tests.test_p6_autonomy tests.test_p7_integrated tests.test_p8_security \
  tests.test_p9_capacity_evidence \
  tests.test_p9_preregistered_v2_integrity -v
```

Résultat : 94/94 tests réussis ; code retour 0. Le log brut se trouve dans `logs/P0_P9_full_regression_v2.txt` et son résumé dans `logs/P0_P9_full_regression_v2.json`. Aucune donnée de modèle ou aucun protocole d'entraînement n'a été modifié par cette suite. Le contrôle du PDF final du présent laboratoire est une étape distincte, pas compris dans les 94 tests.

Interprétation : cette suite vérifie les contrats et les preuves des comportements bornés testés. Elle ne remplace pas des corpus d'évaluation indépendants plus vastes, ni les essais natifs Windows, capteurs physiques ou fournisseurs externes.

Conséquences théoriques/conceptuelles/opérationnelles : les anomalies historiques demeurent dans leurs journaux ; les régressions vertes portent sur l'état courant du code et n'effacent pas les erreurs des premières tentatives. Le PDF courant a été compilé, rendu sur ses 39 pages et contrôlé visuellement sur les pages représentatives ; le journal de revue final est conservé dans `Dmodel_AAM_O93/logs/P9_pdf_final_review.json`.

Conclusion/statut : P0–P9 passent 94/94 tests, y compris l'audit P9 V2, les cinq workers source-off, les checks d'intégrité 14/14 et les tests V2 11/11. P9 est clôturé en `PASS_BOUNDED` après la régression 94/94, les preuves V2 14/14 et 11/11, la compilation réussie et la revue de rendu documentée. P10 demeure distinct : l'acceptation native Windows, les capteurs physiques, les providers Internet réels et les tests NTFS/coupure restent non exécutés.

H.3 P10 – acceptation portable et gates environnementales ouvertes

Description

P10 distingue les comportements que l'hôte actuel permet de tester des critères qui requièrent un Windows natif, un matériel physique ou un fournisseur externe configuré. L'hôte de cette exécution est Linux x86_64, Python 3.13.5 et CPU-only. Les binaires `cmd.exe`, `powershell.exe`, `pwsh`, `wine` et `wine64` sont absents ; aucun périphérique `/dev/video*`, `/dev/snd/*` ou `/dev/hidraw*` n'a été détecté. Ces absences sont les caractéristiques mesurées de l'environnement, et non une conclusion sur AAM.

Architecture et pipeline

Le pipeline de cette gate est : préflight documentaire et de gouvernance ; mesure des capacités de l'hôte ; ajout et inspection statique des lanceurs Windows ; tests portables des chemins Unicode, de l'acquisition CPU, du replay source-off, du cycle autonomie/checkpoint, de l'état réseau par défaut et de l'ouverture différée du matériel ; puis journalisation et revue indépendante du résultat de test. La gate native exige une autre machine, elle n'est pas remplacée par les tests portables.

Commandes exactes

Depuis le répertoire `Dmodel_AAM_O93` sous un environnement Python compatible :

```
PYTHONPATH=. python -m aam_o93.preflight
TERM=dumb PYTHONPATH=. timeout 120s python -m unittest tests.
   test_p10_portable_acceptance -v
python -m py_compile tests/test_p10_portable_acceptance.py
python -m aam_o93 status
```

Le premier contrôle réellement employé appelle `aam_o93.preflight.preflight(root, operation)` et consigne son JSON dans `logs/P10_clean_extract_preflight.json` ; le résultat est `PASS_BOUNDED`. La suite dédiée est consignée dans `logs/P10_portable_acceptance_tests.txt`, son code de sortie capturé vaut 0 et le vérificateur indépendant `logs/P10_test_log_verification.json` confirme cinq tests exécutés, cinq réussis, zéro échec.

Résultats observés

Contrôle	Résultat
Préflight et revue de gouvernance	`PASS_BOUNDED`
Suite portable dédiée	5/5, code retour 0
Chemin avec accents, CJK et emoji	`PASS` sous POSIX
Acquisition CPU, capsule et inférence source-off	`PASS` sur le jeu déclaré
Autonomie, pulse, arrêt, checkpoint et mémoire ON/OFF	`PASS` dans le test dédié
Réseau par défaut	désactivé ; test de fournisseur réel non exécuté
Caméra / audio physique	aucun périphérique détecté ; aucun accès ouvert
Lanceurs Windows `.bat`/PowerShell	fichiers présents, vérifiés statiquement seulement
Installation/désinstallation native Windows, NTFS/ACL, raccourcis	`NOT_RUN`

Le dépôt contenait la documentation du lanceur `run_AAM_O93.bat`, mais pas ce fichier. Cet écart a été corrigé : `run_AAM_O93.bat` et `run_AAM_O93.ps1` sont maintenant présents ; ils se placent dans le dossier du runtime, recherchent Python, vérifient Python 3.11+ et Tk/Tcl, fixent le chemin d'import et propagent le code retour. Les tests locaux confirment leurs contenus et les chemins d'appel, mais leur exécution réelle sur Windows reste non vérifiée.

Interprétation et limites

Les cinq tests établissent un sous-ensemble portable du contrat O93. Ils ne peuvent pas établir à eux seuls la compatibilité avec le système de fenêtres Windows, les ACL/NTFS, les chemins d'installation protégés, les politiques antivirus, ni les pilotes des dispositifs. Aucun vrai flux caméra/audio et aucune requête Internet vers un fournisseur authentifié n'ont été testés. L'application n'ouvre pas les capteurs implicitement ; une permission explicite demeure obligatoire. L'absence de fournisseur ou de matériel ne doit être convertie ni en succès, ni en verdict négatif sur une capacité encore non testée.

Conséquences théoriques, conceptuelles et opérationnelles

Théoriquement, le test de transport logiciel et le test sur plate-forme cible sont deux hypothèses indépendantes. Conceptuellement, « observer » signifie détecter le format et tracer l'origine d'un contenu ; cela ne signifie pas prétendre décoder tous les formats. Opérationnellement, un cycle d'acceptation indépendant doit être effectué sur Windows 10/11 natif avec version de Python notée, Tk/Tcl, droits utilisateur standard, chemins Unicode, installation, première exécution, arrêt/reprise, réinstallation et désinstallation. Les interfaces matérielles doivent faire l'objet d'essais séparés avec autorisation et périphérique identifiés ; le réseau doit être testé contre un fournisseur documenté sans secrets inscrits dans les journaux.

Conclusion et statut

Statut de la gate P10 : `BLOCKED_ENVIRONMENT / PORTABLE_SUBSET_PASS`. La suite portable est réussie (5/5) et les lanceurs sont présents, mais les critères natifs de la gate ne sont pas complets. P10 reste la phase courante ; aucune phase suivante n'est ouverte et O93 n'est pas déclaré intégralement accepté. Les éléments bloqués restent dans `checklist_de_travail_O93.json` avec les preuves et l'environnement qui manque.

Artefacts de preuve

- `logs/P10_host_capability_probe.json`
- `logs/P10_portable_acceptance_tests.txt` et `logs/P10_portable_acceptance_tests.exit`
- `logs/P10_test_log_verification.json`
- `logs/P10_clean_extract_preflight.json` et `logs/P10_compile_final_preflight.json`
- Erreur `O93-E028` : lanceurs Windows absents initialement, corrigés et statiquement vérifiés.
- Incident `O93-ENV036` : discordance de code retour du wrapper, résolue par revue indépendante du journal capturé.
- Incidents documentaires `O93-E029`, `O93-ENV037` et `O93-ENV038` : erreurs de génération TeX corrigées, suivies d'une recompilation et d'un rendu complet.